OpenBench
—User's Guide

Open Source Land Surface Model Benchmarking System

Version 3.0.0b1 (Beta)

CoLM Land Surface Model Development Team

Sun Yat-sen University
School of Atmospheric Sciences

June 2026

Preface

OpenBench (Open Source Land Surface Model Benchmarking System) is an automated, cross-platform framework for evaluating land surface models (LSMs). It collapses the whole workflow —regridding, time alignment, masking, metric computation, plotting, and report generation —that normally has to be hand-assembled for every (model, reference) pair into a single declarative `openbench.yaml` configuration: one config evaluates, compares, and plots any number of variables, models, and reference datasets at once.

This guide accompanies OpenBench 3.0.0b1 (Beta). Functionality is largely complete at the Beta stage, but some interfaces and configuration fields may still change before the final 3.0.0 release; the guide is updated alongside such changes.

June 2026

Contents

Part 1 Introduction

1 Background, Design Philosophy, and Ecosystem

The Land Surface Model (LSM) is one of the core components of Earth system models, describing the water, energy, carbon, and human-activity processes of the land surface. As the number of model versions, parameter schemes, forcing datasets, and resolutions keeps growing, one unavoidable question arises: **how can we objectively and reproducibly evaluate a model's performance against observational/reference data, and make fair comparisons across multiple models?**

The traditional approach is to hand-write a script for every (model, reference) pair: read the data, unify the grid, align time, apply masks, compute metrics, plot, and summarize. This approach has three long-standing pain points:

- **Duplicated effort**: switching to a different variable, model, or reference almost always requires rewriting the entire pipeline.
- **Inconsistent conventions**: different scripts handle regridding, masking, weighting, and missing-value treatment differently, making cross-model comparisons *unfair*.
- **Hard to reproduce**: ad hoc scripts and scattered intermediate files make it difficult to reproduce the same results even months later.

OpenBench (Open Source Land Surface Model Benchmarking System) was created precisely to solve these pain points: it consolidates the entire evaluation pipeline into a single declarative `openbench.yaml` configuration, so that a single configuration can perform preprocessing, evaluation, comparison, and plotting for **any number** of variables, models, and reference datasets, while guaranteeing consistent conventions and reproducible results.

1.1 Design Philosophy and Use Cases

1.1.1 One Configuration, Many Comparisons

OpenBench treats "variable × model × reference" as a Cartesian product: list the variables to evaluate in `openbench.yaml`, bind references, and register models, and OpenBench will

automatically expand all combinations and evaluate them pair by pair (Figure 1). The results of each pair are stored independently and never overwrite one another, so a single configuration can produce a full, cross-referenceable evaluation matrix.

Figure 1: The Cartesian product of variable × simulation × reference expanded into evaluation tasks.

1.1.2 Scientific Consistency and Fair Comparison

The same regridding, masking, weighting, and metric definitions are applied to *every* evaluation pair. This is especially true of the **unified mask**: OpenBench accumulates the missing values of all participating models into a common mask, so that all models are evaluated over *exactly the same* spatial coverage—this is key to the fairness of cross-model comparison and avoids any ”model gaining an advantage because of different coverage.”

1.1.3 Grid and Station on Equal Footing

OpenBench supports **grid** (regular latitude/longitude grid NetCDF) and **station** (per-station observations) references and simulations equally. Station evaluation automatically samples, pairs, and aggregates station by station according to the station list, producing a one-row-per-station metric table.

1.1.4 Incremental, Resumable, and Reproducible

Results are cached by ”configuration + algorithm version + input file fingerprint”: a cache hit is skipped, and a change recomputes only the affected parts; runs can be safely resumed after interruption, without redoing completed work or producing incomplete summaries. Together with the persisted ”fully resolved configuration,” every run is auditable and reproducible.

1.1.5 Typical Use Cases

- Single-model offline evaluation: the overall performance of a new version/new parameter scheme against observations.

- Multi-model intercomparison: answering ”which model is better for which variable” under unified conventions (see Part IV, Case 5).
- Version/sensitivity comparison: the differences between different configurations of the same model.
- Publication-ready plots for papers/reports: one-click generation of publication-quality figures such as Taylor, portrait, and heat-map.

1.2 Relationship to the CoLM Ecosystem and Other Models

OpenBench corresponds to the positioning of ”Volume V: Evaluation System” in the CoLM series, but it is **independent of any specific model under evaluation**: it includes 22 model profiles (CoLM2024, CLM5, NoahMP5, JULES7, VIC5, ERA5-Land, GLDAS2, MATSIRO, CaMa-Flood, WRF, etc.), and can evaluate any LSM output that follows the agreed conventions. In other words, whether you use CoLM, CLM, Noah-MP, or a reanalysis product, you can evaluate and intercompare them with the same OpenBench workflow.

2 Overall Architecture and Execution Mechanism

An OpenBench evaluation proceeds sequentially through four stages, each of which is **incrementally cached and resumable** (Figure 2).

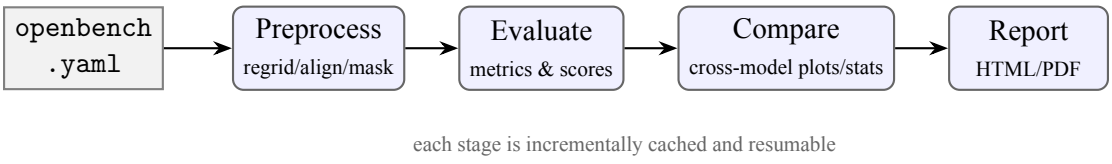


Figure 2: The OpenBench four-stage workflow.

2.1 Stage 1: Preprocess

Each model output and the reference data are regridded to a unified target resolution (`project.grid_res`), cropped to a common time window (`project.years` together with the time-alignment strategy), and a unified mask is applied as needed. The regridding backend is selectable (`regrid_backend`: conservative remapping / CDO / xESMF / simple interpolation).

This stage regularizes inputs that ”look very different from one another” into a unified form that can be compared grid by grid / station by station.

2.2 Stage 2: Evaluate

For each (variable, simulation, reference) triple, the configured metrics and scores are computed: grid evaluation produces per-grid NetCDF (one per metric), and station evaluation produces a one-row-per-station CSV. The default metrics are `bias / RMSE / correlation`, the default score is `Overall_Score`, and both can be configured to a subset or extended to more in the configuration.

2.3 Stage 3: Compare

When `comparison.enabled` is true, shared plots and rankings are generated across all (simulation, reference) pairs: Taylor, portrait, heat-map, SMPI, etc. (by default only Taylor + HeatMap; more must be listed explicitly). Optionally, the `statistics` stage can also run standalone statistical analyses (ANOVA, Mann-Kendall trend, Three-Cornered-Hat, etc.).

2.4 Stage 4: Report

All products are aggregated into an HTML report (a PDF is additionally produced when the `[report]` extra is installed). You can use `generate_report: false` to disable it and produce only data and figures.

2.5 Incremental Caching and Reproducibility Mechanism

Each evaluation task is cached with the key ”configuration + algorithm version + input file signature” (see Part V, Section 24.4):

- **Hit means skip**: the cache is automatically invalidated and recomputed whenever any of the source code (the content fingerprint of the metric/score algorithm), the configuration, or the input files changes.
- **Atomic writes**: all outputs are written to a temporary file and then renamed, so a crash never leaves behind a ”seemingly valid” partial file.

- **Failures do not contaminate**: the post-processing stage has gates, so if any task fails, no potentially misleading partial summary is produced.
- **Resumable**: on a rerun after interruption, completed work is safely skipped and unfinished work is redone; `--force` bypasses the cache for a full recomputation.

3 Core Concepts, Terminology, and Capabilities

Table 1: Core terminology used throughout the book

Term	Meaning
variable	The standard physical quantity being evaluated (e.g., Evapotranspiration, Latent_Heat)
reference	The benchmark dataset used for evaluation (grid or stn), from the built-in registry or self-registered
simulation	A model output entry being evaluated (label → path and variable mapping)
profile	A template of a model's variable-name/unit/file-name conventions, referenced by a simulation
metric	A per-grid/per-station error measure (bias, RMSE, KGE⋯)
score	A comparable score normalized to 0–1 (larger is better), such as Overall_Score
comparison	Cross-model/reference summary plots and rankings (Taylor, portrait, heatmap⋯)
groupby	Re-aggregating results by IGBP/PFT/Köppen categories
unified mask	A NaN mask accumulated across models, ensuring consistent spatial coverage
time_alignment	The matching strategy for the model and reference time windows (intersection/per_pair/strict)
regrid	Interpolating data of different resolutions onto a unified target grid

3.1 Capability Overview

4 Versioning, Compatibility, and Reading Guide

Note: Applicable version for this manual: OpenBench 3.0.0b1 (Beta)

Main differences from v2.x: a single `openbench.yaml` replaces the old multi-file Fortran namelists; the command line is unified into `openbench` subcommands

Table 2: OpenBench capability overview

Dimension	Content
Reference datasets	**101** built-in (grid + station), supporting list / show / scan / register
Model profiles	**22** built-in (CoLM2024, CLM5, NoahMP5, JULES7, VIC5, ERA5-Land, GLDAS2, MATSIRO, CaMa-Flood, WRF⋯)
Metrics	**25+**: bias, percent_bias, RMSE, ubRMSE, CRMSD, MAE, correlation, R^2, NSE, KGE, KGESS, index_agreement, rv⋯
Scores	**8** normalized scores: nBiasScore, nRMSEScore, nPhaseScore, nIavScore, nSpatialScore, nSeasonalityScore, index_agreement, Overall_Score
Statistics modules	**17**: ANOVA, Correlation, Mann-Kendall, Three-Cornered-Hat, Hellinger, PLSR, FDR⋯
Comparison visualizations	Taylor, Target, Portrait, HeatMap, Parallel Coordinates, KDE, Ridgeline, Whisker, Diff, SMPI, Relative Score, Mann-Kendall⋯
Classification groupings	IGBP (17 classes), PFT (16 classes), Köppen (30 classes), with masks bundled in the package
Interfaces	CLI (run/check/init/ref/sim/model/cache/migrate/smoke-test/gui) + an optional GUI wizard

(run/check/init/ref/sim/model⋯); evaluation results are **numerically consistent** with v2.x.

What about old configurations: use `openbench migrate old.nml -o openbench.yaml` (or `.json`) for one-click conversion, see Part IV, Case 4.

This guide accompanies OpenBench 3.0.0b1 (**Beta**). The Beta-stage features are essentially complete, but some interfaces and configuration fields may still change before the official 3.0.0 release. The version number follows PEP 440: 3.0.0b1 means ”the 1st Beta of 3.0.0,” and once finalized the suffix is dropped to become 3.0.0. The evaluation results of OpenBench 3.0 are numerically consistent with 2.x.

Note: Installing the Beta pre-release: while only the Beta is on PyPI, `pip install colm-openbench` will install it directly; once the official version is released, installing the Beta requires `pip install --pre colm-openbench` or explicitly specifying the version number.

4.1 Structure of This Guide and Its Audience

This guide is divided into five parts, organized by reader level:

- **Part I ·Introduction** (this part): concepts, architecture, and terminology, to establish an overall understanding.
- **Part II ·Quick Start**: for first-time users—installation, troubleshooting of supporting software, self-checks, and running your first evaluation in five minutes.
- **Part III ·Model Configuration**: a configuration reference manual that explains, block by block, all fields and values of `openbench.yaml`.
- **Part IV ·Hands-on Examples**: reproducible end-to-end cases (including a real multi-model comparison) that you can copy and adapt by replacing the data and models as needed.
- **Part V ·Developer's Guide**: for users who need to extend functionality, integrate their own data/models, perform secondary analyses, or contribute to development.

Beginning users should start from Part II; long-term users can use Part III as a reference manual; developer users should focus on Part V. The chapters are linked to one another via cross-references (e.g., "see Section 9.5"), making it easy to jump around.

Part 2 Quick Start

This part is aimed at first-time users: first prepare the software environment and install the package, work around common pitfalls with ”heavy” dependencies such as Cartopy, run a self-check with the bundled sample, then run your first evaluation in five minutes, and finally get to know the output artifacts.

5 Software Environment and Installation

OpenBench is a pure-Python package that runs across Linux / macOS / Windows and requires **Python 3.10 or higher**. The core scientific stack is fairly ”heavy”, so we recommend installing it in a clean virtual environment (venv / conda) to avoid conflicts with the system Python.

Table 3: Dependency overview

Category	Contents
Core (installed automatically)	xarray, numpy, scipy, pandas, netCDF4, matplotlib, cartopy, dask, joblib, flox, PyYAML, Jinja2, click, tqdm, packaging
`[gui]`	PySide6 (graphical configuration wizard)
`[remote]`	paramiko, cryptography (SSH remote execution)
`[report]`	xhtml2pdf (PDF reports; without it, only HTML is produced)
`[migration]`	f90nml (migrating legacy Fortran NML configs)
`[all]`	all of the optional extras above

Note: For HPC/servers, the core installation (CLI only) is sufficient; `[gui]` is not needed. When an optional dependency is missing, OpenBench gives a friendly hint rather than crashing (for example, if `[report]` is not installed it automatically skips PDF and produces only HTML).

5.1 Installation

OpenBench is published as the Python package `colm-openbench`, installable via **pip** (§5.1.2) or **conda / conda-forge** (§5.1.3). The core scientific stack is fairly ”heavy”, so we recommend installing it in a clean virtual environment (venv or conda) to avoid polluting the

system Python. **If your platform lacks prebuilt packages for geospatial libraries such as Cartopy, we recommend using conda directly** (see §5.1.3), which avoids local compilation.

> **Note: Command notation (used throughout this book)**: in code blocks, anything after `#` is a *comment* for explanation only and **should not be typed**; `<...>` (angle brackets) is a *placeholder* to be replaced with your own value (for example, replace `<work_dir>` with a real path); whereas `[...]` (square brackets) **are part of the command and must be entered verbatim**—this is pip's "extras (optional dependencies)" syntax: `pip install "colm-openbench[gui]"` means installing the core package *together with* the additional group of optional dependencies named `gui`. When square brackets are present, wrap the whole argument in quotes `"..."` so that shells like zsh do not treat the brackets as glob patterns and error out.

5.1.1 Creating a virtual environment

```
python -m venv .venv                    # create an isolated virtual
    environment under .venv/
source .venv/bin/activate               # activate it (Windows: .venv\
    Scripts\activate)
python -m pip install --upgrade pip     # upgrade pip itself while you're
    at it (recommended)
```

5.1.2 Installing from PyPI (pip)

The simplest—install core only (command-line evaluation, best suited for HPC / headless servers):

```
pip install colm-openbench              # install core CLI only, without
    any extras
```

When you need extra features, append the "extras" in square brackets after the package name (they can be combined with commas, e.g. `[gui,report]`):

```
pip install "colm-openbench[gui]"         # + graphical configuration
    wizard (needs PySide6/Qt)
pip install "colm-openbench[remote]"      # + remote SSH execution (needs
    paramiko)
pip install "colm-openbench[report]"      # + PDF/HTML reports (needs
    xhtml2pdf)
```

```
pip install "colm-openbench[migration]"  # + legacy Fortran namelist
    migration (needs f90nml)
pip install "colm-openbench[all]"        # + all of the above
pip install "colm-openbench[gui,report]" # you can also pick a few, comma
    -separated, no spaces
```

For the specific dependencies introduced by each ”extra”, see the dependency overview earlier (Table 3, §5). Choose as needed: for HPC/pure command line, core is enough; for a graphical interface add `[gui]`; for finished reports add `[report]`; for everything use `[all]`.

> **Note:** This is currently the Beta pre-release version 3.0.0b1. If both a stable release and a Beta exist on PyPI, `pip` installs the stable release by default; to explicitly install the Beta, use `pip install --pre colm-openbench` or `pip install colm-openbench==3.0.0b1`.

5.1.3 Installing with conda / conda-forge

OpenBench ships its own conda-forge recipe (the repository file `conda/meta.yaml`, `noarch`: it pulls the source from PyPI, and conda resolves geospatial dependencies such as Cartopy/NetCDF/PROJ). conda users have three paths:

(a) Install directly from conda-forge (after release). Once it is officially available on conda-forge, a single command suffices—conda installs the ”heavy” dependencies such as Cartopy/NetCDF/PROJ along with it, sparing you local compilation:

```
conda install -c conda-forge colm-openbench   # installs the heavy
    geospatial deps too
mamba install -c conda-forge colm-openbench   # or use the faster mamba (
    equivalent)
```

> **Warning:** This is currently the Beta pre-release version 3.0.0b1, which is **not yet available on conda-forge**. Until it is, please use option (b) below (”conda for dependencies + pip for the package itself”), or option (c) to build locally from the bundled recipe.

(b) conda resolves dependencies + pip installs the package itself (recommended for Beta). If your platform has no prebuilt wheel for Cartopy and the like, first install the geospatial scientific stack with conda-forge, then install OpenBench with pip (optionally with extras such as `[gui]`):

```
mamba create -n openbench -c conda-forge python=3.12 cartopy netcdf4  #
    create env + install geospatial deps
mamba activate openbench                          # activate the
    environment
pip install colm-openbench                        # then install the
    package itself with pip (or "colm-openbench[all]")
```

(c) Build a local conda package from the bundled recipe (optional). If you want a local conda package right now, build it using the recipe in the repository:

```
conda install -n base conda-build              # install the conda-build
    tool
conda build conda/                             # build locally using
    conda/meta.yaml
conda install -c local colm-openbench          # install the just-built
    package from the local channel
```

5.1.4 Installing from source (development / local testing)

```
git clone https://github.com/CoLM-SYSU/OpenBench.git  # clone the source
    repository
cd OpenBench                                          # enter the
    repository directory
python -m pip install -e ".[dev]"                     # editable install
    + dev dependencies (pytest/ruff)
```

Note: The classification masks needed for category groupings (IGBP/PFT/Köppen) ship with the package (about 1.1 MB compressed), so no extra download is required.

5.2 Surrounding Software and Dependency Troubleshooting

Land-surface benchmarking involves several ”heavy” scientific libraries, and the most common installation/runtime problems center on Cartopy and NetCDF. This section gives solutions you can follow directly.

5.2.1 Cartopy system libraries (GEOS / PROJ) build failures

Cartopy depends on the system-level GEOS and PROJ libraries. If `pip install` reports compilation errors, install the system dependencies first and then reinstall:

```
# Debian/Ubuntu
sudo apt-get install -y libgeos-dev libproj-dev proj-bin

# macOS (Homebrew)
brew install geos proj
```

Alternatively, install Cartopy directly with conda (see the previous section) to skip local compilation.

5.2.2 Cartopy / Natural Earth map-data download errors and manual download

Symptom. The installation itself succeeds, but during the *plotting stage* (comparison / figures), the first time base-map features such as coastlines, land, and national borders are needed, Cartopy downloads Natural Earth vector data on demand over the network. On offline, intranet, or firewalled machines (especially HPC compute nodes), this throws an error similar to the following and aborts plotting:

On a machine *with network access*, it appears as download notices (the download succeeds and plotting continues):

```
.../cartopy/io/__init__.py:241: DownloadWarning: Downloading:
  https://naturalearth.s3.amazonaws.com/110m_physical/ne_110m_land.zip
.../cartopy/io/__init__.py:241: DownloadWarning: Downloading:
  https://naturalearth.s3.amazonaws.com/110m_physical/ne_110m_coastline.
     zip
.../cartopy/io/__init__.py:241: DownloadWarning: Downloading:
  https://naturalearth.s3.amazonaws.com/110m_physical/ne_110m_lakes.zip
.../cartopy/io/__init__.py:241: DownloadWarning: Downloading:
  https://naturalearth.s3.amazonaws.com/110m_physical/
     ne_110m_rivers_lake_centerlines.zip
```

On an *offline / intranet / firewalled* machine, the same download fails and aborts plotting, with an error such as:

```
urllib.error.URLError: <urlopen error [Errno 110] Connection timed out>
# or
HTTPError: HTTP Error 404: Not Found
# or
ssl.SSLCertVerificationError: certificate verify failed
```

Cause. This is not an OpenBench bug, but rather Cartopy's mechanism of fetching base-map resources on demand at runtime failing in an environment without internet access. By

default, OpenBench base-map requests target exactly the four `110m_physical` datasets above (`land`, `coastline`, `lakes`, `rivers_lake_centerlines`).

Solution: download manually and place them in the Cartopy data directory.

Step 1, determine the data directory Cartopy expects:

```
python -c "import␣cartopy;␣print(cartopy.config['data_dir'])"
# common paths:
#   Linux/macOS:  ~/.local/share/cartopy
#   can also be specified via the CARTOPY_DATA_DIR environment variable
```

Step 2, on a machine with network access, download the four `110m_physical` datasets needed by OpenBench's default base map (when evaluating finer base maps as needed, you can likewise add the `50m`/`10m` versions). Cartopy's default source:

```
https://naturalearth.s3.amazonaws.com/110m_physical/ne_110m_land.zip
https://naturalearth.s3.amazonaws.com/110m_physical/ne_110m_coastline.zip
https://naturalearth.s3.amazonaws.com/110m_physical/ne_110m_lakes.zip
https://naturalearth.s3.amazonaws.com/110m_physical/
   ne_110m_rivers_lake_centerlines.zip
official page: https://www.naturalearthdata.com/downloads/
```

Step 3, unzip and place them in the `physical` subdirectory (the directory structure must match the resolution/category):

```
DATA_DIR=$(python -c "import␣cartopy;␣print(cartopy.config['data_dir'])")
DEST="$DATA_DIR/shapefiles/natural_earth/physical"
mkdir -p "$DEST"
for n in ne_110m_land ne_110m_coastline ne_110m_lakes
   ne_110m_rivers_lake_centerlines; do
  unzip -o "$n.zip" -d "$DEST"
done
```

Step 4 (optional), you can also use Cartopy's bundled download script to fetch everything at once (on a machine with network access):

```
python -m cartopy.feature.download physical cultural --output "$DATA_DIR"
```

> **Note:** After they are in place, running OpenBench again will produce figures during the plotting stage without any network access. If you use a shared `CARTOPY_DATA_DIR`, the whole cluster can reuse a single copy of the base maps.

5.2.3 netCDF4 / HDF5 related issues

Two common kinds of issues:

(1) HDF5 file locking (NFS / shared storage). On some NFS or cluster shared filesystems, HDF5 file locking causes errors or hangs when reading NetCDF (something like `HDF5 error: unable to lock file`). Temporarily disable file locking:

```
export HDF5_USE_FILE_LOCKING=FALSE
openbench run openbench.yaml
```

(2) numpy / netCDF4 ABI mismatch. If on import you get a binary-compatibility warning/error such as `numpy.ndarray size changed`, it is usually because the numpy version does not match netCDF4/the compiled extension. Reinstall to align them:

```
pip install --force-reinstall --no-cache-dir numpy netCDF4
```

5.2.4 Other optional dependencies (PySide6 / f90nml / xhtml2pdf)

These dependencies are needed only when you use the corresponding feature; when missing, OpenBench gives a clear hint:

- **PySide6** (`[gui]`): required to run `openbench gui`. If missing, you are prompted to `pip install "colm-openbench[gui]"`.
- **f90nml** (`[migration]`): required to migrate legacy `.nml` configs (`openbench migrate xxx.nml`); not required for migrating JSON.
- **xhtml2pdf** (`[report]`): required to generate PDF reports. Without it, the evaluation still completes normally and only an HTML report is produced, with no error.

6 Verifying the Installation and First Run

After installation, before preparing your own data, we strongly recommend first running a self-check with the **bundled sample**—it does not depend on any of your data and confirms in one step that ”it is installed and runs end to end”:

```
openbench smoke-test          # unpack the bundled Initial_test sample
   and run openbench check (fast)
```

```
openbench smoke-test --run    # additionally run a full evaluation (
    including figures)
openbench smoke-test --run --keep   # keep the working directory after
    the run, for inspecting artifacts
```

The command prints the working directory and config path; at the end of `--run` it also lists the key result artifacts generated (and verifies they were indeed produced). After the run, under `output/openbench_initial_test_smoke/` in the working directory, focus on these places:

- **Overall-score heatmap**: `comparisons/HeatMap/scenarios_Overall_Score_comparison_heatma` (with companion data `..._comparison.csv`).
- **Spatial distribution map**: `scores/<variable>_..._Overall_Score.jpg` (gridded).
- **Station results**: `<variable>_stn_..._evaluations.csv` under `metrics/` and `scores/` (one row per station).

If `smoke-test` passes, the environment is ready. A full walkthrough of this sample appears in Part IV, Case 1.

> **Note:** The full evaluation of `--run` may, on a clean machine and during the first plotting, trigger Cartopy's network download of base maps (see Section 5.2.2).

6.1 Run Your First Evaluation in Five Minutes

An overview of all OpenBench commands (`openbench --help`):

```
Commands:
  run          Run evaluation from a config file.
  check        Validate config file and check data availability.
  smoke-test   Run the bundled Initial_test smoke fixture.
  init         Interactively generate an openbench.yaml config file.
  ref          Manage reference datasets.
  sim          Manage simulation outputs.
  model        Manage model profiles.
  migrate      Convert old JSON/NML config to unified YAML.
  cache        Manage OpenBench runtime caches.
  gui          Launch the OpenBench graphical interface.
  version      Show version information.
```

6.1.1 Before running: prepare your ref and sim data first

The smoke-test in §6 already verified the environment with the bundled sample; to evaluate **your own model**, you **must first prepare two kinds of data** on the local machine:

- **Reference data**: observations/benchmarks, as gridded NetCDF or station data.
- **Simulation data**: the model output you want to evaluate.

OpenBench itself *does not include* these data; it only ships a registry describing "how to read known datasets"—so **you must provide the data yourself**, and the `init` below merely *discovers, registers, and generates the config* (it does not copy data).

> **Warning: Do you need to scan separately first? No.** `openbench init` *asks you on the spot* for the "reference data root" and the "simulation data root", then **automatically scans and registers them internally** (equivalent to running `ref scan` / `sim scan` for you). The standalone `openbench ref scan <dir>` / `openbench sim scan <dir>` are for *hand-written configs, batch/CI pre-registration, or rescanning newly added data later*—going through `init` for the first time does not require a separate scan. If during scanning you encounter **unrecognizable data** (ambiguous variables, non-standard layouts, etc.), see Part V, Section 22.4.1 for how to handle it.

To get started, you only need the three steps `init` → `check` → `run`: generate config → validate → run.

```
openbench init                    # 1) interactively generate config: asks
    you for the ref root + sim root
openbench check openbench.yaml    # 2) validate config + data availability
openbench run   openbench.yaml    # 3) evaluate + plot + report
```

What each step does:

- `init`: an interactive wizard—**it first asks you for the "reference data root" and the "simulation data root"**, automatically scans and registers the datasets there, then in turn lets you select variables, pick a reference for each variable, and confirm the model profile, generating a ready-to-run `openbench.yaml` (if you prefer not to be interactive, you can hand-write it or assemble it with `ref scan`/`sim scan`; see Parts III and V).
- `check`: **do this before running**. It reports item by item whether each (variable, reference) was located successfully, how many evaluation tasks were expanded in total, and

the value of each option; it can catch path/variable-name/unit problems before the run actually starts.

- `run`: in turn completes preprocessing, evaluation, comparison, and reporting. Re-running after editing the config recomputes only the affected parts; `--force` recomputes everything.

Results land in `output/<project_name>/`; open `reports/report.html` first.

6.1.2 Common options for init / check / run

Common options of the three core commands:

`openbench init` (interactively generate config, with options to preset scanning behavior):

Option	Meaning
`--ref-root PATH`	reference data root (for init scanning/update precheck)
`--refresh-ref`	refresh the reference catalog directly during init, without asking
`--no-ref-check`	skip the reference catalog scanning/update precheck
`--sim-root PATH`	simulation data root to scan (repeatable)
`--sim-model NAME`	the model profile name for the simulation, or `auto` (default auto)
`--sim-case-depth N`	directory depth for scanning simulation cases (0–10, default 5)
`--sim-case-pattern GLOB`	include only case labels matching this glob
`--sim-exclude GLOB`	simulation directory names/globs to exclude (repeatable)
`--sim-climatology auto|off|year|month`	detect or force the simulation's climatology `tim_res` handling

Note: Does init rescan the reference every time? No. Whether it scans depends on the state of the reference catalog:

- **First time** (catalog missing or empty): **a scan is mandatory**; this is a prerequisite for init to continue.
- **Already registered** (catalog non-empty): init **does not rescan automatically**; it only asks "Reference catalog exists. Refresh it from the data root now?", **defaulting to "No"**—*pressing Enter skips the scan and reuses the registered references*.

The registration result is persisted in the user directory at `~/.openbench/references/reference_catalog.ya` and the reference root is remembered too, so every subsequent init can reuse it without rescanning. To control this behavior: `--refresh-ref` forces a rescan/refresh (use it when the data has been updated), `--no-ref-check` skips the reference precheck entirely (use it when already registered and you want to reach config generation as fast as possible), and `--ref-root PATH` specifies the reference root directly.

`openbench check`:

- `--comparison-only`: validate in "comparison-only" mode, skipping the local simulation-root check.
- `--strict-reference` (`--strict`): treat low-confidence reference metadata as errors (does not modify the YAML).
- `--variable VAR` (repeatable): validate only the specified variable.

`openbench run`: `--force` (ignore the cache and recompute everything), `--comparison-only` (re-run only the comparison on existing results), `--dry-run` (check only, do not execute), `--cores N`, `--variable VAR` (repeatable), `--output-dir DIR`, `--dump-config` (export the intermediate/debug config).

6.2 Output Artifacts at a Glance

A successful `run` produces roughly the following (see the cases in Part IV and the results interpretation in Section 7 for details):

```
output/<project_name>/
+-- config.yaml          # the full config actually used this run (with
    resolved defaults)
+-- run.log              # the complete run log
+-- data/                # preprocessed sim/ref NetCDF (after regridding
    and clipping)
+-- metrics/             # one NC(grid)/CSV(station) + figure per (
    variable,reference,simulation,metric)
+-- scores/              # normalized scores (named like metrics) +
    figures
+-- comparisons/         # cross-model comparison figures and data (when
    comparison.enabled=true)
+-- statistics/          # statistical analysis (when statistics.enabled
    and items are present)
```

```
+-- reports/              # report.html (also report.pdf when [report] is
    installed)
`-- .openbench_cache.json # incremental cache index
```

Recommended viewing order: `reports/report.html` → `comparisons/` (cross-model overview) → `metrics/`, `scores/` (pair-by-pair detail).

7 FAQ and Troubleshooting Run Failures

- **`pip install colm-openbench` cannot find a version?** This is currently a Beta pre-release; use `pip install --pre colm-openbench` or specify `==3.0.0b1` explicitly.
- **Cartopy won't install / plotting reports a Natural Earth download error?** See Sections 5.2.1 and 5.2.2.
- **Reading NetCDF reports an HDF5 file-locking error?** `export HDF5_USE_FILE_LOCKING=FALSE` (see Section 5.2.3).
- **`check` reports that a variable has no reference?** Check the variable-name case in `reference`, whether the dataset name exists (`openbench ref show <name>`), and whether `data_root`/the paths are correct.
- **Don't have your own data and want to try it first?** Just run `openbench smoke-test --run --keep` (the bundled sample).
- **Does groupby (IGBP/PFT/climate zones) require an extra mask download?** No; the masks ship with the package.

7.1 Troubleshooting Run Failures

`check` is a **static** validation (config/paths/data reachability); passing it does not guarantee that `run` will succeed—the run may still fail due to data content, plotting, memory, etc. Common scenarios and remedies:

> **Note:** The first step in troubleshooting is always to look at the `run.log` in the output directory—it records, stage by stage, each task's success/failure and the reason.

Symptom	Remedy
`check` passes but `run` fails	Look at the error at the end of `run.log`; usually a variable-name/unit mismatch, the file's actual content lacking the variable, or out of memory (lower `num_cores` or enable `dask`).
A comparison sub-item fails (e.g. Portrait)	Usually insufficient data coverage (e.g. too few finite values in a season); OpenBench reports the single item as failed, produces the rest as usual, and exits non-zero (see Case 3). Complete the data or remove that figure type.
No figures after the run	Check that `comparison.enabled: true` and that `items` are explicitly listed; the pair-by-pair figures are under `metrics/` and `scores/`.
Plotting stage hangs / reports a download error	Cartopy failed to fetch base maps offline—place Natural Earth manually per Section 5.2.2.
Stations all skipped / not found	The `DIR` path in the station CSV is wrong or relative; `min_year_threshold` is too large and filters out short-record stations.
Edited the config but nothing recomputed	What you edited is not part of the fingerprinted input; use `--force` or delete that run's `.openbench_cache.json` (see Part V, Section 24.4).
Region/station/global results don't match	The three use different conventions: `lat_range`/`lon_range` determine the region, stations are paired point by point, and the weight (`weight`) affects aggregation—make sure the conventions are consistent before comparing.
HDF5 file-locking error (NFS)	`export HDF5_USE_FILE_LOCKING=FALSE`.

Part 3 Configuration

This part is the configuration reference manual for OpenBench, walking through every field, value, and common pitfall of `openbench.yaml` block by block. Notation: a field name is written as `block.field`; when a field is omitted, OpenBench substitutes the *default* noted below; in code blocks, `<...>` denotes a placeholder.

8 Overall Configuration Structure and a Complete Example

8.1 Configuration format: YAML (replacing the legacy namelist)

OpenBench 3.x describes a single evaluation with **one YAML file** (`openbench.yaml`), replacing the 2.x-era multi-file Fortran **namelist** (`.nml`) scheme. How the two correspond: the `general`/`ref`/`sim`/`metrics` fields that were scattered across the old namelist are now consolidated into the eight YAML top-level blocks described below. An existing namelist (or JSON) legacy configuration can be converted to YAML in one step with `openbench migrate` (see Part IV, Example 4). This part covers only the new YAML configuration; if you still maintain a legacy namelist, migrate it first and then follow this manual.

8.2 The eight top-level blocks

The skeleton of a configuration is shown below, with each of the eight top-level blocks playing its own role:

```
project:      # Global: identifier, spatial-temporal extent, target
   resolution, parallelism, alignment, mask, weighting, report
evaluation:   # Which variables to evaluate
reference:    # data_root + which reference dataset(s) each variable is
   bound to
simulation:   # The model output(s) being evaluated (one or more, label
   -> entry)
metrics:      # Which metrics to compute        (omitted -> [bias, RMSE,
   correlation])
scores:       # Which normalized scores to compute (omitted -> [
   Overall_Score])
comparison:   # Cross-model comparison plot toggle and items (items
   omitted -> [Taylor_Diagram, HeatMap])
```

```
statistics:   # Statistics analysis toggle and items (items omitted ->
   empty set, i.e. nothing runs even if enabled)
```

Of these, `project`, `evaluation`, `reference`, and `simulation` are **required**; the other four may be omitted and fall back to defaults.

8.3 _defaults: merging shared settings

Writing a `_defaults` block under `simulation` merges its fields into every simulation entry, avoiding repetition:

```
simulation:
  _defaults:
    data_type: grid
    tim_res: Month
    data_groupby: Month
  run_a: {model: CoLM2024, root_dir: /data/A}
  run_b: {model: CLM5,     root_dir: /data/B}
```

8.4 !include: splitting across files

Split a section of configuration into a separate file and pull it in with `!include`, for easier reuse and version management:

```
simulation: !include sims/all_models.yaml
```

> **Note: The deprecated `options` block**: fields written under a top-level `options:` in legacy configurations are automatically migrated by OpenBench into `project` with a warning. In new configurations, write them directly under `project`, or run `openbench migrate`.

8.5 A complete main configuration example

Below is a main `openbench.yaml` that is **complete across all eight blocks and annotated**; copy it as a template and modify as needed. The remaining chapters of this part explain each field block by block. This template has passed validation by the OpenBench configuration loader and is shipped as `examples/openbench-full.yaml`.

OpenBench ships with 4 example configurations that **serve different purposes; read the labels carefully before running** (the same label also appears at the top of each file):

Table 4: Bundled example configurations

File	Status	Description
`openbench-smoke.yaml`	**Runnable** (smoke-tested)	Built-in sample template; run it with `openbench smoke-test --run`, do not manually run the raw file with placeholders
`openbench-full.yaml`	Template only / needs user data	Annotated reference template complete across all eight blocks
`openbench-min.yaml`	Template only / needs user data	Minimal configuration skeleton
`multimodel.yaml`	Needs user data / placeholder paths	Four-model comparison (Example 5); paths must be replaced

Warning: Except for `openbench-smoke.yaml` (driven by `smoke-test`), the paths in the other examples are all **placeholders**; running `run` on them directly will fail because the data cannot be found—replace them with local paths first.

```yaml
project:
  # --- Identifier (required) ---
  name: my_benchmark                  # Output lands in output_dir/name/
  output_dir: ./output
  years: [2004, 2005]                 # Evaluation time window, inclusive
     of endpoints
  # --- Spatial-temporal extent ---
  lat_range: [-90, 90]
  lon_range: [-180, 180]
  min_year_threshold: 1               # Station: drop stations with valid
     years < N
  # --- Target resolution and alignment ---
  tim_res: Month                      # Year|Month|Day|Hour|climatology-
     month|climatology-year
  grid_res: 2.0                       # Target resolution to regrid to (
     degrees)
  time_alignment: intersection        # intersection | per_pair | strict
  regrid_backend: openbench_conservative
  # --- Runtime and fairness ---
  num_cores: 4                        # null/0 -> all CPUs
  weight: area                        # area | mass | none (omitted is
     also area)
  unified_mask: true
  generate_report: true
  # --- Group-by (default false) ---
```

```
  IGBP_groupby: false
  PFT_groupby: false
  climate_zone_groupby: false
  # --- Optional performance sub-blocks (defaults shown; usually no need
     to change;
  #     see the "Performance tuning and environment variables" chapter)
     ---
  io:
    netcdf_compression: false        # zlib-compress output NetCDF?
    netcdf_compression_level: 1
    mfdataset_batch_size: null       # null=auto batching; 0=no batching
  dask:
    enabled: false                   # enable the dask.distributed
       runtime
    threads_per_worker: 1
    memory_limit: auto

evaluation:
  variables: [Evapotranspiration, Latent_Heat]

reference:
  data_root: /data/Reference
  Evapotranspiration: ET_Xu_etal_2025_LowRes   # Single reference
  Latent_Heat: FLUXCOM                         # Can also be a list for
     multi-reference comparison

simulation:
  _defaults:                         # Merged into each entry below (
     optional)
    data_type: grid
    tim_res: Month
    data_groupby: Month
  CoLM2024_run:                      # Label = name shown in outputs/
     legends (your choice)
    model: CoLM2024                  # Matches the built-in profile ->
       automatic variable mapping
    root_dir: /data/Simulation/CoLM2024
    prefix: hist_                    # Files <prefix>YYYY-MM.nc
    variables:                       # Override the profile's variable
       names/units
      Latent_Heat: {varname: f_lfevpa, varunit: W m-2}
  CLM5_run:                          # Multi-model comparison: just add
     more entries
    model: CLM5
    root_dir: /data/Simulation/CLM5
    prefix: clm5_hist_
  NoahMP5_run:
    model: NoahMP5
```

```
    root_dir: /data/Simulation/NoahMP5
    prefix: noahmp5_
  ERA5Land_run:                       # A reanalysis can also participate
     as a "model"
    model: ERA5-Land
    root_dir: /data/Simulation/ERA5-Land
    prefix: era5land_

metrics: [bias, RMSE, correlation, KGE]    # Omitted -> [bias, RMSE,
   correlation]
scores:  [nBiasScore, nRMSEScore, Overall_Score]   # Omitted -> [
   Overall_Score]

comparison:
  enabled: true
  items: [HeatMap, Taylor_Diagram]   # Omitting items yields exactly
     these two

statistics:
  enabled: false                      # Enabling requires enabled: true
     and an explicit items list
```

9 The project block: global settings

The `project` block gathers the **global settings** of an evaluation: identifier and output, spatial-temporal extent, target resolution and alignment, runtime and fairness. The fields are described below in four groups (required fields are noted in each group).

9.1 Identifier and output

Table 5: Identifier fields (required)

Field	Value	Meaning
`name`	String	Project name; output lands in `output_dir/name/`
`output_dir`	Path	Output root directory (relative to cwd or absolute)
`years`	`[start, end]`	Evaluation time window, inclusive of endpoints, exactly two integers

9.1.1 `name`

Project name, which determines the output subdirectory `output_dir/name/`. It must be a *single directory name* (no path separators or spaces); rerunning with the same name reuses the same output directory and triggers incremental caching (changing the configuration recomputes only the affected parts). Changing `name` yields a fresh, independent output that does not interfere with others.

9.1.2 `output_dir`

The output root directory, which may be relative (to the current working directory) or absolute. All products (data/metrics/scores/figures/comparisons/reports) land in the `name/` directory beneath it.

9.1.3 `years`

Two integers of the form `[start, end]`, **inclusive of endpoints**. This is the "requested range" of the evaluation time window; the actual range used is determined after intersecting it with each dataset's available years and with `time_alignment`. If a dataset does not cover the requested years at all, that pair is skipped and noted in the log/check output.

9.2 Spatial-temporal extent

Table 6: Spatial-temporal extent fields

Field	Default	Meaning
`lat_range`	`[-90,90]`	Latitude range
`lon_range`	`[-180,180]`	Longitude range
`min_year_threshold`	3	Station references: drop stations with valid years $<$ N (stations only)

9.2.1 `lat_range` / `lon_range`

Latitude/longitude ranges of the form `[min, max]`, defaulting to global (`[-90,90]` / `[-180,180]`). OpenBench uses these to clip *all* gridded data and stations (stations outside the

range are excluded). For a regional evaluation (e.g. East Asia only, or a single basin), just narrow these two ranges without altering the data itself. The longitude convention depends on the data ($[-180, 180]$ or $[0, 360]$); just keep it consistent with the data.

9.2.2 `min_year_threshold`

Defaults to 3 and **applies only to station references**. If a station's valid observation period is shorter than this threshold, the station is dropped and excluded from evaluation. This filters out stations with overly short, statistically unstable records; lowering it (e.g. to 1) retains more short-record stations (the built-in sample uses 1), while raising it is stricter.

9.3 Target resolution and alignment

This group of fields determines the "common scale" to which all data is unified, affecting *all* stages—evaluation, comparison, and statistics.

Table 7: Resolution and alignment fields

Field	Value / Default	Meaning
`tim_res`	`Year`, `Month`, `Day`, `Hour`, `climatology-month`, `climatology-year` (also `3Hour`/`6Hour`/`8Day`)	Target time resolution (incl. **climatology**; see "Climatology evaluation mode")
`grid_res`	Float (degrees), e.g. 0.5	Target spatial resolution (regrid to this)
`timezone`	Float (hours)	Timezone offset (meaningful only at hourly/daily scales). **Not yet implemented**: accepted but currently has no effect
`time_alignment`	`intersection` (default)	See below
`regrid_backend`	`openbench_conservative` (default)	See below

9.3.1 `tim_res`

The target **time** resolution to which all data is aggregated/aligned. Valid values: `Year`, `Month`, `Day`, `Hour`, `3Hour`, `6Hour`, `8Day` (plus multi-month forms such as `3month`), plus two **climatology** values `climatology-month` and `climatology-year` (see "Climatology evalua-

tion mode" below). It must match the evaluation intent: for a monthly evaluation, write `Month`. If the source data is finer than the target, OpenBench aggregates in time; if coarser, it may be unable to satisfy the request and raises an error.

Climatology evaluation mode Setting `tim_res` to `climatology-year` or `climatology-month` enters **climatology** evaluation— the entire time span is first averaged (duration-weighted) into a climatology and then compared with the reference, rather than evaluating the time series at each time step:

- `climatology-year`: **annual climatology**—the entire time span is averaged into a *single* time point (multi-year mean state).
- `climatology-month`: **monthly climatology**—grouped by month and averaged into *12* time points (multi-year mean seasonal cycle).

```
project:
  years: [2001, 2010]
  tim_res: climatology-month     # Evaluate the multi-year mean seasonal
     cycle
```

When to use: when you care about how the model represents the *long-term mean state* or the *mean seasonal cycle* (rather than point-by-point agreement of the month-by-month time series).

> **Warning:** Under climatology, some metrics that **depend on the time dimension** are unavailable: the annual climatology has only 1 time point, so correlation, trend, phase, etc. cannot be computed; OpenBench automatically skips these metrics and notes "Unsupported climatology metric(s)" in the log. The monthly climatology has 12 points, so seasonal correlation and the like can be computed, but interannual-variability metrics still do not apply. Choose metrics according to your evaluation goal.
>
> **Note:** Time resolution (including climatology) is specified uniformly in v3 by `project.tim_res`, which also drives the internal `compare_tim_res` of the comparison stage. The legacy practice of writing `tim_res`/`grid_res`/`weight` etc. under `comparison` is **deprecated**—OpenBench still migrates them automatically into `project` with a warning; it is recommended to write them directly under `project` or to run `openbench migrate`.

9.3.2 `grid_res`

The target **spatial** resolution (in degrees, e.g. 0.5, 0.25, 2.0). All gridded data is **regridded** to this resolution before comparison—this is the key to putting models/references of different native resolutions onto the same grid. Generally choose the coarser of the parties being compared, to avoid forcibly interpolating coarse data onto an overly fine grid. Station evaluation is unaffected by this (paired per station).

9.3.3 `timezone`

Timezone offset (in hours, float). It is meaningful only for **hourly/daily** evaluation, where it aligns the model and observation times to the same timezone; at monthly scale and above it can be ignored.

> **Warning: This field is not yet implemented in the current version**: the value is accepted and recorded but **no** actual time shift is applied, so it has no effect on results. If you need cross-timezone alignment, handle it in the source data for now; a future release will add this.

9.3.4 Time alignment (`time_alignment`)

One of three choices, determining how the model and reference time windows are matched:

- `intersection` (default): evaluate over the **intersection** of the two time windows. Most robust, recommended for most scenarios.
- `per_pair`: each (sim, ref) pair aligns **independently**, producing one aligned copy of the ref for each sim. Suitable when different sims have substantially different temporal coverage and you want each to make the most of its own.
- `strict`: requires the time windows to be **exactly identical**, otherwise raises an error. Suitable for rigorous scenarios that require strictly comparable data.

9.3.5 Regrid backend (`regrid_backend`)

One of four choices, determining how data is interpolated to `grid_res`:

- `openbench_conservative` (default): OpenBench's built-in conservative remapping; requires no external dependencies, preserves flux conservation, and is suitable for **flux-type** variables such as ET, latent heat, and precipitation. It is the recommended default.
- `cdo_remapcon`: calls the system CDO's conservative remapping (requires CDO installed). Usable when you want consistency with an existing CDO workflow.
- `xesmf_conservative`: calls xESMF's conservative remapping (requires xESMF/ESMF installed).
- `basic_interpolation`: simple (bilinear, etc.) interpolation, the fastest but *not conservative*; suitable for state variables or quick trial runs, but use with caution for flux-type variables.

9.4 Runtime and fairness

Table 8: Runtime fields

Field	Value / Default	Meaning
`num_cores`	Integer / `None`	Number of parallel cores (`None`/0 → all CPUs)
`weight`	`area|mass|none`	Spatial aggregation weight, **omitted → area**
`unified_mask`	`true` (default)	Cross-model accumulated unified mask (fairness)
`generate_report`	`true` (default)	Produce HTML/PDF report

9.4.1 `num_cores`

Number of parallel cores. `None` (omitted) or 0 means use all CPUs; setting a specific integer limits usage (explicitly setting it is recommended on shared HPC nodes). It controls variable/task-level parallelism; for finer distributed scheduling see `dask`.

9.4.2 `weight` (aggregation weight, three values)

Controls how per-grid results are weighted when *aggregated into a single number*. It does **not** change the per-grid results under `metrics/` and `scores/`; it *only* affects downstream aggregation (such as the "Overall" column of grouped tables):

- `area` (**the actual default when omitted**): cosine-latitude area weight $\cos\phi$, corresponding to the true grid-cell area. The **correct choice for global/regional evaluation**.
- `mass`: area weight $\times$ flux weight (using the time-mean absolute value of the *reference* data as the flux weight), giving higher weight to "regions with large flux"—suitable for flux variables such as ET, GPP, and precipitation.
- `none` (**only when written explicitly**): simple arithmetic mean, all valid grid cells equally weighted (high latitudes overweighted).

  Warning: Counterintuitively: *omitting* `weight` $\rightarrow$ actually uses `area`; only *explicitly* writing `weight: none` gives the arithmetic mean. The two give different results—most users do not write this field, meaning they are using area weighting.

9.4.3 `unified_mask`

Defaults to `true`. It **accumulates** the missing data of each model being compared into a common mask, so that all models are evaluated over *exactly the same* spatial coverage. This is the key to fairness in cross-model comparison—always keep it on for multi-model comparison; when evaluating a single model its effect is minor.

9.4.4 `generate_report`

Defaults to `true`, generating an HTML report (also a PDF when `[report]` is installed). Setting it to `false` skips the report and produces only data and figures, suitable for batch/scripted scenarios.

9.4.5 Advanced switches

- `debug_mode` (default false): prints diagnostic information during the data-processing stage, for troubleshooting; it is not a global log level.
- `only_drawing` (default false): **skips evaluation** and re-renders figures using existing results only—very useful when you change plotting-related settings and want to quickly re-render figures.

- `force` (default false): bypasses incremental caching and **recomputes everything** (equivalent to the command-line `--force`).
- `strict_reference` (default false): upgrades "low-confidence" (LOW provenance) references from a warning to an error, forcing the use of trusted references only.

9.5 Group-by: IGBP / PFT / Köppen

When enabled, evaluation results are additionally aggregated **by land cover / climate category**:

```
project:
  IGBP_groupby: true          # 17-class MODIS/IGBP land cover
  PFT_groupby: true           # 16-class plant functional types
  climate_zone_groupby: true  # 30-class Köppen climate zones
```

The three masks (`IGBP.nc`, `PFT.nc`, `Climate_zone.nc`) are **shipped with the package** (about 1.1 MB compressed) and work out of the box. To use a custom / higher-resolution mask, override via an environment variable or the run directory (which take higher priority):

```
export OPENBENCH_DATASET_DIR=/path/to/your/dataset   # Highest priority
# Or place ./dataset/<name>.nc in the run directory
```

Group-by results land in `output/<run>/comparisons/{IGBP,PFT,Climate_zone}_groupby/`. For an example see Part IV, Example 3.

10 evaluation and reference: variables and reference data

10.1 evaluation.variables

Lists the **standard variable names** to evaluate (non-empty). A variable name is OpenBench's internal unified "standard name"; the original variable names of each model/reference are mapped to it via the `varname` of the profile/registry:

```
evaluation:
  variables: [Evapotranspiration, Latent_Heat, Sensible_Heat]
```

10.2 Available standard variable names

The built-in reference registry covers 70+ standard variable names, grouped by domain below (use `openbench ref show <dataset>` to check which variables a given dataset provides):

- **Energy/radiation**: `Latent_Heat`, `Sensible_Heat`, `Ground_Heat`, `Net_Radiation`, `Surface_Net_SW_Radiation`, `Surface_Net_LW_Radiation`, `Surface_Downward_SW_Radiation`, `Surface_Downward_LW_Radiation`, `Surface_Upward_SW_Radiation`, `Surface_Upward_LW_Radiation`, `Surface_Albedo`
- **Evapotranspiration/hydrology**: `Evapotranspiration`, `Transpiration`, `Canopy_Evaporation`, `Canopy_Transpiration`, `Bare_Soil_Evaporation`, `Open_Water_Evaporation`, `Precipitation`, `Runoff`, `Total_Runoff`, `Streamflow`, `Surface_Soil_Moisture`, `Root_Zone_Soil_Moisture`, `Soil_Moisture_Lev2`, `Terrestrial_Water_Storage_Change`, `Inundation_Area`, `Inundation_Fraction`
- **Carbon/ecology**: `Gross_Primary_Productivity`, `Net_Primary_Production`, `Net_Ecosystem_Exchange`, `Ecosystem_Respiration`, `Biomass`, `Leaf_Area_Index`, `Burned_Area`, `FCH4`, `Methane`, `Wetland_Methane_Emission`
- **Meteorology**: `Surface_Air_Temperature`, `Diurnal_Max_Temperature`, `Diurnal_Min_Temperature`, `Surface_Soil_Temperature`, `Root_Zone_Soil_Temperature`, `Surface_Pressure`, `Surface_Relative_Humidity`, `Surface_Specific_Humidity`, `Surface_Wind_Speed` (including `_U10`/`_V10`), `Cloud_Cover`, `Lake_Surface_Water_Temperature`, `Ground_Frost_Frequency`
- **Snow/ice**: `Snow_Depth`, `Snow_Water_Equivalent`, `Snow_Cover_Extent`
- **Urban/human activity**: `Urban_Air_Temperature_Max`/`_Min`, `Urban_Albedo`, `Urban_Surface_Temperature`, `Urban_Latent_Heat_Flux`, `Urban_Anthropogenic_Heat_Flux`, `Dam_Inflow`/`_Outflow`, `Dam_Water_Storage`/`_Elevation`, `Crop`, `Total_Irrigation_Amount`, `Water_Use`

Note: Variable names are **case-sensitive** and must match the registry/profile exactly. Whether a variable can be evaluated depends on whether a reference dataset and a model profile both provide it.

10.3 reference: data_root + variable binding

`reference` contains a common `data_root` and a "variable → reference source" mapping. A variable can be bound to a **single** or to **multiple** reference sources (list form); when multiple, each is evaluated independently (sim × ref Cartesian product):

```
reference:
  data_root: /data/Reference
  Evapotranspiration: ET_Xu_etal_2025_LowRes          # Single reference
  Latent_Heat: [FLUXCOM, ILAMB_monthly]               # Multiple
     references, each evaluated once
```

10.4 Dataset registry and commands

Reference datasets come from the built-in registry (101 of them). Common commands:

```
openbench ref list                              # List all reference datasets
openbench ref show ET_Xu_etal_2025_LowRes   # View a dataset's details/
   variables
openbench ref scan /data/Reference          # Scan a directory and auto-
   register existing data
```

Note: Many datasets have multi-resolution versions (suffixes `_LowRes`/`_MidRes`). A base name (e.g. `GLEAM_v4.2a`) resolves to a "resolution family", or you can directly specify the concrete suffixed name.

10.5 The two reference types: grid and station

Reference data is divided by `data_type` into `grid` (gridded NetCDF) and `stn` (station: per-station NetCDF + a station-list CSV). Station references are governed by `project.min_year_threshold`: stations with an insufficient valid period are dropped. To register your own data see Part V, Section 22.

10.5.1 Station-list CSV format

Station data (reference or simulation) uses a `fulllist` CSV that points each station to its NetCDF. The header columns are:

```
ID,SYEAR,EYEAR,LON,LAT,Flag,DIR
```

- `ID`: station identifier; `SYEAR`/`EYEAR`: the station's start/end year; `LON`/`LAT`: longitude/latitude.

- `Flag`: whether to include (true/false); `DIR`: the path to the station's NetCDF (absolute or relative).

You can use `openbench ref generate-station-list` to auto-generate this list from a batch of station NetCDFs (see Part V, Section 22).

11 The simulation block and model profile

11.1 Entry structure

`simulation` is a "label → entry" dictionary that can hold multiple models being evaluated. The entry fields are listed in Table 9.

Table 9: `simulation.<label>` fields (verified against `config.schema.SimulationEntry`)

Field	Value	Meaning
`model`	String (required)	Model profile name (e.g. CoLM2024), resolves the variable mapping
`root_dir`	Path (required)	Model output root directory
`data_type`	`grid|stn`	Data form (overrides the profile)
`grid_res` / `tim_res`	Float / String	Spatial/time resolution (overrides the profile)
`data_groupby`	`Year|Month|Day|single`	Time-splitting granularity of the output files
`prefix` / `suffix`	String	Filename prefix/suffix
`fulllist`	Path	Station-list CSV (for station simulations)
`variables`	Mapping	Per-variable overrides: `varname`, `varunit`, `prefix`, etc.

11.2 Field values in detail

11.2.1 `model`

Model profile name (e.g. `CoLM2024`). When it matches a built-in profile, that profile's variable names/units/ file conventions are applied automatically (see the subsection below); a non-match also works, as long as you fully specify each variable's `varname` under `variables`.

11.2.2 `root_dir`

The root directory (absolute or relative) where this model's output files reside. OpenBench searches here (or its subdirectories) by `prefix`/`suffix`.

11.2.3 `data_type`

`grid` (regular lat-lon grid NetCDF) or `stn` (station: per-station NetCDF + station list). Determines whether this model is paired with the reference by grid or by station for evaluation.

11.2.4 `data_groupby`

The **time-splitting granularity** of the output files, which determines how OpenBench locates files by year:

- `Year`: one file per year (the filename contains the year).
- `Month`: one file per month (the filename contains YYYY-MM, etc.).
- `Day`: one file per day.
- `single`: all time steps in *one* file.

11.2.5 `grid_res` / `tim_res`

The native resolution of this model's data, overriding the profile default. All of it is ultimately regridded/ aggregated to `project`'s target resolution.

11.2.6 `prefix` / `suffix`

Filename prefix and suffix; OpenBench matches by `{prefix}{year}*{suffix}.nc`. A suffix starting with a dot must be quoted (`suffix: ".021"`).

11.2.7 `fulllist`

The path to the station-list CSV for a station simulation (containing ID, longitude/latitude, and per-station file path). Required when `stn`.

11.2.8 `variables`

A per-variable override mapping; each variable can set `varname` (the variable name inside the NetCDF), `varunit` (unit, used for conversion), and a **per-variable prefix** (to handle models that "split files by variable", see below and Example 5).

11.3 The model profile matching mechanism

When `model` matches a built-in profile (22 of them), the variable filename patterns and unit conversions are **applied automatically**; fields within the entry **override** the profile. To view a profile's variable mapping: `openbench model show CoLM2024`.

11.4 File naming: prefix / suffix / data_groupby

OpenBench searches for a given year's file by `{prefix}{year}*{suffix}.nc`, with `data_groupby` indicating the splitting granularity:

```
simulation:
  CoLM2024_run:
    model: CoLM2024
    root_dir: /data/CoLM2024
    data_groupby: Month            # Files <prefix>YYYY-MM.nc
    prefix: hist_
    variables:
      Latent_Heat: {varname: f_lfevpa, varunit: W m-2}
```

11.5 Per-variable overrides and "splitting files by variable"

`variables.<var>` can override `varname`/`varunit`, and can also set a **per-variable prefix**—which is key for models where "each variable is in its own file with the variable name encoded in the filename" (such as TE):

```
    variables:
      Latent_Heat:   {varname: LTNT, prefix: "YEE2_JRA-55_LTNT_M"}
      Sensible_Heat: {varname: SENS, prefix: "YEE2_JRA-55_SENS_M"}
```

For a complete four-model integration example see Part IV, Example 5.

> **Warning:** A suffix starting with a dot (e.g. `.021`) is treated as a number by YAML and must be quoted as `suffix: ".021"`.

11.6 Auto-generating the simulation configuration with sim scan

Hand-writing each model's `prefix`/`suffix`/`data_groupby` is error-prone. `openbench sim scan` can scan a model's output directory, automatically infer these fields, and generate a `simulation` snippet that you can paste directly into the configuration:

> **Note: The directory conventions of sim and reference differ**: reference scanning relies on *fixed variable-name folders* (`Grid/<resolution>/<category>/<variable>/<dataset>`, see Part V, Section 22.4); whereas **simulations have no variable-folder convention**—`sim scan` **discovers "cases"** within `--case-depth` (default 5) levels (one case = one model run directory), and **the directory name becomes the case label** (for a directory named `history/output`, its parent directory name is taken instead); as for variable names/units, those are mapped by the **model profile** and are *not* inferred from folder names. So simulation data can be organized into directories however you like; the key is to give the right `--model`.

```
# First preview which cases were found and what was inferred (writes
    nothing)
openbench sim scan /data/Simulation --model auto --dry-run

# Once confirmed, write out the simulation configuration snippet (
    including station lists)
openbench sim scan /data/Simulation --model auto \
    --output sims.yaml --report scan_report.yaml -y
```

Common options: `--model auto` (auto-match the profile, or specify one such as `CoLM2024`), `--case-depth N` (scan directory depth), `--case-pattern GLOB` / `--exclude GLOB` (filter/exclude cases), `--climatology auto|off|year|month`, `--station-output DIR` (output directory for station lists/merged files). The generated `sims.yaml` can be pulled into the main configuration with `!include` (`simulation: !include sims.yaml`), then validated with `openbench check`.

> **Note:** `openbench init` internally also calls the same scanning logic (its `--sim-*` options correspond to those here, see Part II, Section 6.1.2).

11.7 What to do when sim scan cannot infer the model profile

`sim scan` must determine for each case "which **model profile** produced it" (which decides variable names/units/filename conventions). When a case **cannot be auto-matched** to a known profile, it is marked `unresolved`. This differs from the reference scan's "unrecognized layout"— the way to handle it is to **give it a model profile**:

- **Specify a profile**: `sim scan ... --model CoLM2024` (force the use of a registered profile); within `openbench init` it will instead *prompt you to enter* a profile name and then rescan automatically.
- **Register the model first**: if the model has no profile yet, add one with `openbench model register` / `model import`, or use `sim scan ... --register-model NAME` (optionally with `--overwrite-model`) to **generate a draft profile from the scan metadata**.
- **Exclude irrelevant directories**: `--exclude GLOB` (repeatable) drops subdirectories that are not model output.
- **Pass through now, fill in later**: `--allow-unresolved` permits writing unresolved cases into the YAML as well (with `model` left empty), to be filled in manually later and then validated with `openbench check`.

A complete example. When `--model auto` cannot infer the model for the case `MyRun`, it errors out and provides handling hints:

```
$ openbench sim scan /data/Simulation --model auto --dry-run
Simulation scan reported unresolved cases:
- model inference: MyRun
  Specify --model MODEL for an existing profile, create a draft with
  --model NewModel --register-model, or re-run with --allow-unresolved.
  For exact mappings, use `openbench model register NewModel -v StdName:
     nc_var:unit`.
Pass --allow-unresolved to publish anyway.

# A) A profile for this model already exists -- specify it directly
$ openbench sim scan /data/Simulation --model CoLM2024 -o sims.yaml -y
Wrote sims.yaml

# B) No profile -- generate a draft profile from the scan metadata (named
    MyModel)
$ openbench sim scan /data/Simulation --model MyModel --register-model -o
    sims.yaml -y
```

```
Wrote draft model profile .../model_catalog.yaml
Wrote sims.yaml

# C) Pass through first -- write into the YAML with model left empty,
    fill in manually later
$ openbench sim scan /data/Simulation --model auto --allow-unresolved -o
    sims.yaml -y
```

The draft profile (option B) is written into the user overlay's `model_catalog.yaml`, which you can then inspect with `openbench model show MyModel` and complete with precise variable mappings via `model register MyModel -v Latent_Heat:LH:W m-2`.

It is advisable to first run `--dry-run` to see which cases resolve successfully or fail, then decide. For handling "unrecognized" cases on the reference side see Part V, Section 22.4.1.

12 Configuring metrics, scores, comparison, and statistics

`metrics` and `scores` are top-level lists referenced by method name. When omitted they default respectively to `[bias, RMSE, correlation]` and `[Overall_Score]`. When you only care about certain items, list them explicitly:

```
metrics: [bias, RMSE, ubRMSE, correlation, KGE, NSE]
scores:  [nBiasScore, nRMSEScore, Overall_Score]
```

12.1 Metrics described item by item

OpenBench has 25+ built-in metrics, grouped by nature in Table 10 (s=simulation, o=observation/reference; in the range column, the value after $\to$ is the "optimal value").

Table 10: Metrics described item by item

Metric	Range	Meaning
Bias-type		
`bias`	$(-\infty, \infty) \to 0$	Mean bias $\overline{s} - \overline{o}$
`percent_bias`	$\% \to 0$	Relative bias $100\,(\overline{s} - \overline{o})/\overline{o}$
`absolute_percent_bias`	$\% \to 0$	Absolute value of the relative bias
Error-type		

Metric	Range	Meaning
`RMSE`	$[0, \infty) \to 0$	Root-mean-square error
`ubRMSE`	$[0, \infty) \to 0$	Unbiased RMSE $= \sqrt{\text{RMSE}^2 - \text{bias}^2}$, looks only at the amplitude of variation
`CRMSD`	$[0, \infty) \to 0$	Centered RMSD (same concept as ubRMSE, used in Taylor diagrams)
`mean_absolute_error`	$[0, \infty) \to 0$	Mean absolute error
Correlation-type		
`correlation`	$[-1, 1] \to 1$	Pearson correlation coefficient
`correlation_R2`	$[0, 1] \to 1$	Squared correlation coefficient (fraction of explained variance)
`ubcorrelation` / `ubcorrelation_R2`	As above	Unbiased versions
Overall efficiency-type		
`NSE`	$(-\infty, 1] \to 1$	Nash−Sutcliffe efficiency (0 = equal to the mean, <0 worse than the mean)
`KGE`	$(-\infty, 1] \to 1$	Kling−Gupta efficiency (combines correlation/bias/variance ratio)
`KGESS`	$(-\infty, 1] \to 1$	KGE skill score (relative to a reference baseline)
`ubNSE` / `ubKGE`	$(-\infty, 1] \to 1$	Unbiased versions
`index_agreement`	$[0, 1] \to 1$	Willmott index of agreement
Variance / amplitude / other		
`rv`	$[0, \infty) \to 1$	Variance ratio σ_s/σ_o
`kappa_coeff`	$[-1, 1] \to 1$	Categorical agreement (suitable for categorical variables)
`pc_max` / `pc_min` / `pc_ampli`	%	Relative bias of the maximum/minimum/amplitude
`rm_mean`	—	Residual measure after removing the mean
`smpi`	$[0, \infty) \to 0$	Single-model performance index (normalized mean-square difference)

Metric	Range	Meaning
`br2`, `cp`, `dr`, `APFB`, `L`, `MFM`	—	Specialized/advanced metrics, formulas in the source `core/metrics.py`

Note: `rSD`, `PBIAS_HF`, and `PBIAS_LF` are reserved interfaces that currently raise `NotImplementedError` explicitly, so do not use them in `metrics`.

12.2 Scores described item by item

Scores normalize metrics to $[0, 1]$ (higher is better), making them easier to compare across variables/models:

Table 11: The 8 normalized scores

Score	Meaning
`nBiasScore`	Normalized score based on bias
`nRMSEScore`	Normalized score based on RMSE
`nPhaseScore`	Phase agreement (degree of seasonal-cycle alignment)
`nIavScore`	Interannual variability agreement
`nSpatialScore`	Spatial distribution agreement
`nSeasonalityScore`	Seasonality agreement
`index_agreement`	Willmott index of agreement (also usable as a score)
`Overall_Score`	**Composite score** (weighted synthesis of the above), the core input to portrait/ranking

12.3 Configuring comparison and statistics

12.3.1 comparison: cross-model comparison plots

```
comparison:
  enabled: true                  # Default false
  items:                         # Omitted -> [Taylor_Diagram, HeatMap] (
     not all)
    - Taylor_Diagram
    - HeatMap
    - Portrait_Plot_seasonal
    - Single_Model_Performance_Index
```

Warning: Omitting `items` produces only the *two* default plots (Taylor + HeatMap), **not** all of them. To produce more plot types you must list them explicitly.

The available plot types described item by item (Table 12):

Table 12: Available comparison plot types

Plot type	Description
`HeatMap`	Composite heatmap of all metrics/scores across (sim, ref), showing the overall picture at a glance
`Taylor_Diagram`	Polar plot combining correlation coefficient, normalized standard deviation, and CRMSD
`Target_Diagram`	(bias, ubRMSE) scatter plot, showing the combination of bias and dispersion
`Portrait_Plot_seasonal`	Portrait heatmap, one cell per (item, ref, sim) $\times$ 4 seasons
`Single_Model_Performance_Index`	SMPI composite score + bootstrap confidence, the data source for ranking
`Kernel_Density_Estimate`	sim/ref probability density comparison (distribution shape)
`Parallel_Coordinates`	Parallel-coordinates plot over multiple metrics, a multi-dimensional overview
`Whisker_Plot`	Box-and-whisker plot over multiple metrics (distribution and outliers)
`Ridgeline_Plot`	Ridgeline distribution plot (multiple distributions stacked)
`Diff_Plot`	sim $-$ ref difference map
`Relative_Score`	Relative score (relative to a baseline)
`Mann_Kendall_Trend_Test`	Trend significance test plot
`Correlation`	Cross-model correlation analysis comparison
`Standard_Deviation`	Standard-deviation comparison (amplitude of variability)
`Functional_Response`	Comparison of functional responses between variables

Plot type	Description
`RadarMap`	Radar plot (overview of multiple metrics × multiple models)

Note: When `enabled: false`, no comparison products are produced regardless of what `items` contains; with only 1 sim the comparison plots degenerate into single points/single lines and are not very meaningful (multi-model comparison is where the value lies). Portrait requires finite data across all four seasons × each metric; when data coverage is insufficient it errors on the individual item and skips it (see Part IV, Example 3).

12.3.2 statistics: standalone statistical analysis

```
statistics:
  enabled: true                    # Default false
  items:                           # Omitted -> empty set (nothing runs
     even if enabled!)
    - ANOVA
    - Correlation
    - Mann_Kendall_Trend_Test
    - Hellinger_Distance
```

Warning: Unlike comparison: `statistics.enabled: true` with `items` omitted → is equivalent to not enabling it. To run the statistics module you must have **enabled + an explicit items list**.

The available statistics methods described item by item (the 16 options exposed on the GUI statistics page, Table 13):

Table 13: Available statistics methods

Method	Description
Descriptive / aggregation	
`Basic`	Summary of basic statistics
`Mean` / `Median`	Time-dimension mean / median
`Min` / `Max` / `Sum`	Time-dimension minimum / maximum / sum
`Standard_Deviation`	Standard deviation (amplitude of variability)

Method	Description
`Z_Score`	Standardized score (degree of anomaly)
Relationship / attribution	
`Correlation`	Correlation analysis
`Partial_Least_Squares_Regression`	Partial least squares regression (multi-predictor attribution)
`Functional_Response`	Functional-response relationship between variables
Trend / distribution / error decomposition	
`ANOVA`	Analysis/decomposition of variance
`Mann_Kendall_Trend_Test`	Non-parametric trend significance test
`Hellinger_Distance`	Similarity between two distributions
`False_Discovery_Rate`	FDR control for multiple hypothesis testing
`Three_Cornered_Hat`	Three-cornered-hat error decomposition (requires 3 data sources)

Note: `Three_Cornered_Hat` is only meaningful with 3 independent data sources. The underlying `core/statistics/` also contains several internal modules (such as Covariance, Variance, Autocorrelation, etc.) mainly for programmatic use, which are not among the GUI options in the table above. Statistics-stage products land in `output/<run>/statistics/<method>/`.

13 Performance tuning and environment variables

For high-resolution grids or large numbers of files, tune `project.num_cores` and the two optional sub-blocks `project.io` and `project.dask`. Both sub-blocks can be omitted entirely (falling back to defaults).

13.1 The io sub-block: high-throughput IO (all fields)

13.2 The dask sub-block: distributed scheduling (all fields)

13.3 Configuration example

Table 14: All `project.io` fields (verified against `config.schema.IOConfig`)

Field	Default	Meaning
`netcdf_compression`	`false`	Whether to enable zlib compression for numeric NetCDF output
`netcdf_compression_level`	1	Compression level (1 = fast default)
`mfdataset_batch_size`	`None`	Multi-file read batch size: `None`/auto = automatic by the resource planner; 0 = disable batching; N = N files per batch
`mfdataset_auto_batch_min_files`	`None`	Minimum number of files to trigger auto-batching
`mfdataset_auto_batch_min_size_mb`	`None`	Minimum total volume (MB) to trigger auto-batching
`mfdataset_auto_batch_min_size`	`None`	Minimum batch size for auto-batching (byte-level threshold)
`mfdataset_auto_batch_max_size`	`None`	Maximum batch size for auto-batching (upper limit on file count)
`mfdataset_auto_batch_memory_fraction`	`None`	Fraction of memory batching may use (e.g. 0.25)

Table 15: All `project.dask` fields (verified against `config.schema.DaskConfig`)

Field	Default	Meaning
`enabled`	`false`	Whether to enable dask.distributed scheduling
`scheduler`	`None`	Existing scheduler address (e.g. `tcp://...`); if empty, start one locally
`n_workers`	`None`	Number of workers (empty = automatic)
`threads_per_worker`	1	Threads per worker
`processes`	`true`	Use multiple processes (true) or multiple threads (false)
`memory_limit`	`auto`	Per-worker memory limit (e.g. `4GB`, `auto`)
`dashboard_address`	`None`	Dask dashboard address (e.g. `:8787`)
`local_directory`	`None`	Worker spill temporary directory

```yaml
project:
  num_cores: 16
  io:
    netcdf_compression: true
    netcdf_compression_level: 1
    mfdataset_batch_size: auto
    mfdataset_auto_batch_min_files: 200
    mfdataset_auto_batch_memory_fraction: 0.25
  dask:
    enabled: true
    n_workers: 4
    threads_per_worker: 1
    processes: true
    memory_limit: auto
    dashboard_address: ":8787"
```

Note: Environment variables **take priority over** YAML, which is convenient for ad-hoc benchmarking, for example:

```
OPENBENCH_NETCDF_COMPRESSION=1 OPENBENCH_DASK=1 OPENBENCH_DASK_N_WORKERS=4
openbench run openbench.yaml --force
```

13.4 Environment variable reference

OpenBench recognizes a set of `OPENBENCH_*` environment variables, used to **temporarily override** configuration or to specify data/cache locations. Environment variables **take priority over** YAML, which suits batch processing, HPC, and benchmarking (change the environment once, without touching the configuration file).

Note: There are also a few *advanced/internal* variables (such as `OPENBENCH_ALGORITHM_VERSION`, `OPENBENCH_STATION_MATCHER_JOBS`, `OPENBENCH_ROOT_MARKERS`, etc.) that ordinary users need not set.

14 The GUI configuration wizard (optional)

If the `[gui]` extra is installed, you can use the graphical wizard to interactively generate a configuration and manage runs:

```
openbench gui          # Requires pip install "colm-openbench[gui]" first
```

Table 16: Common environment variables (verified against the source)

Variable	Meaning
Data / paths	
`OPENBENCH_DATASET_DIR`	Directory for group-by masks (IGBP/PFT/Köppen), overriding the bundled version
`OPENBENCH_REF_ROOT`	Reference data root directory (overrides the root in the registry/configuration)
`OPENBENCH_DATA_ROOT`	General data root directory
`OPENBENCH_SIM_ROOT`	Simulation data root directory
`OPENBENCH_CUSTOM_DIR`	Custom filter/extension directory
`OPENBENCH_INCLUDE_ROOTS`	Search roots for `!include`
IO (corresponds to project.io)	
`OPENBENCH_NETCDF_COMPRESSION` / `_COMP_LEVEL`	NetCDF compression switch / level
`OPENBENCH_MFDATASET_BATCH_SIZE`	Multi-file read batch size
`OPENBENCH_MFDATASET_AUTO_BATCH_*`	Auto-batching thresholds (MIN_FILES / MIN_SIZE / MIN_SIZE_MB / MAX_SIZE / MEMORY_FRACTION)
Dask (corresponds to project.dask)	
`OPENBENCH_DASK`	Enable dask scheduling
`OPENBENCH_DASK_N_WORKERS` / `_THREADS_PER_WORKER`	Number of workers / threads per worker
`OPENBENCH_DASK_MEMORY_LIMIT` / `_PROCESSES`	Memory limit / process mode
`OPENBENCH_DASK_SCHEDULER` / `_DASHBOARD_ADDRESS` / `_LOCAL_DIRECTORY`	Scheduler address / dashboard / spill directory
Regrid weight cache	
`OPENBENCH_REGRID_WEIGHT_CACHE_DIR`	Regrid weight cache directory
`OPENBENCH_REGRID_WEIGHT_CACHE_MAX_GB` / `_TTL_DAYS`	Cache size limit / time-to-live
External libraries	
`HDF5_USE_FILE_LOCKING`	Set `FALSE` to disable HDF5 file locking (NFS/shared disks)
`CARTOPY_DATA_DIR`	Cartopy basemap data directory (shared basemap)

14.1 Interface layout

The wizard is paginated **by configuration block**, corresponding one-to-one with the chapters of this part:

- **General / Runtime**: project's identifier, spatial-temporal extent, resolution, alignment, parallelism, weighting, etc.
- **Evaluation / Reference Data**: select variables and bind a reference dataset to each variable.
- **Simulation Data**: register model output (sim scan can be invoked to scan).
- **Metrics / Scores / Comparisons / Statistics**: check metrics, scores, comparison plot types, and statistics methods.
- **Registry**: manage the reference/model registry (register, view).
- **Preview**: preview the generated `openbench.yaml`.
- **Run Monitor**: run and view progress/logs.

14.2 Typical operations

1. **New**: fill in each page → **Preview** to view the YAML → **Save** to export `openbench.yaml`.
2. **Load an existing one**: **Load Config** opens an existing YAML, the pages are auto-populated, and you can continue editing.
3. **Validation**: **Validate/Check** is equivalent to `openbench check`, flagging data/field issues one by one.
4. **Run**: **Run** executes on the **Run Monitor** page and shows progress in real time; remote runs are also configured here (see Part V, Section 24).

What the GUI exports is a standard `openbench.yaml`, **fully interoperable** with the command line (a configuration saved by the GUI can be run with `openbench run`, and vice versa).

14.3 Common issues

- `openbench gui` reports a missing PySide6 → `pip install "colm-openbench[gui]"`.

- A headless server (pure SSH) cannot pop up a window → use the command-line workflow; the GUI suits local workstations.
- Validate reports that a variable has no reference → same as `check`, troubleshoot per Part V, Section 22.6.

Part 4 Worked Examples

This part demonstrates the complete workflow—from organizing data, writing configuration, validating, running, to interpreting results—through reproducible, end-to-end examples. Example 1 is based on the `Initial_test` sample bundled with OpenBench, so it can be reproduced with a single command on any machine after installation; the later examples show extended comparisons, connecting your own data, multi-model intercomparison, and migration of legacy configurations. Overview of the examples in this part:

Table 17: Overview of the examples in this part

Example	Content	Data
1	Initial_test: full workflow for evapotranspiration / energy flux (grid + station)	Bundled sample (real run)
2	Substitute your own data (register references, connect models, station lists)	Hands-on guide
3	Extended comparison plots (Taylor) + IGBP class grouping	Bundled sample (real run)
4	Migrate from a legacy JSON/NML configuration and run	Hands-on guide
5	Intercomparison of four real land-surface models (CoLM2024/CLM5/TE/GLDAS)	Real model data (real run)

Suggested reading path: first follow Example 1 to run the bundled sample end-to-end and build an intuitive sense of the outputs and the workflow; then follow Example 2 to swap in your own data; when you need multi-model intercomparison, refer to Example 5. For examples marked "real run", the table values and figures all come from actual `openbench run` output.

15 Example 1: Bundled Evapotranspiration / Energy-Flux Evaluation (Initial_test)

15.1 One-command reproduction

OpenBench ships with a small evaluation sample, `Initial_test`, and provides the `smoke-test` command to automatically unpack the data, generate the configuration, and run it:

```
# Validate the configuration and data only (fast)
openbench smoke-test

# Run the full evaluation and keep the work directory so you can inspect
    the products
openbench smoke-test --run --keep
```

When the command finishes it prints the **work directory** and the **config path**; with `--run` it also lists the **key result products** (the overall-score heat map and the spatial distribution maps), for example:

```
Smoke fixture: /tmp/openbench-smoke-XXXX/Initial_test
Config:        /tmp/openbench-smoke-XXXX/openbench-smoke.yaml
...
Smoke result artifacts:
  Total score heatmap:
    - comparisons/HeatMap/scenarios_Overall_Score_comparison_heatmap.jpg
  Total score heatmap data:
    - comparisons/HeatMap/scenarios_Overall_Score_comparison.csv
  Overall score spatial maps:
    - scores/Evapotranspiration__ref__..._sim__grid_case__Overall_Score.
       jpg
    - ...
Smoke run passed.
Kept smoke work directory: /tmp/openbench-smoke-XXXX
```

`smoke-test --run` **verifies that these result products are actually generated** (the overall-score heat map plus the Overall_Score spatial map for each pair); if any are missing it raises an error—so it validates both "installed and runnable" and "can produce result figures". This section uses that sample to walk through the configuration, the validation output, and the result products segment by segment. It also covers both the **grid** and **station** evaluation modes.

15.2 Example overview and data organization

This sample evaluates the output of 1 model against several reference datasets, over the period 2004–2005, at monthly resolution and a 2° grid:

- **Variables**: evapotranspiration Evapotranspiration, latent heat Latent_Heat, sensible heat Sensible_Heat
- **Simulation** (two forms of the same CoLM output): `grid_case` (grid) and `station_case` (about 58 flux stations)

- **References**: GLEAM v4.2a (grid), GLEAM_hybrid_PLUMBER2 (station), ILAMB-FLUXCOM (grid), PLUMBER2 (station)

The directory layout after unpacking is as follows:

```
Initial_test/
+-- Reference/Initial_test/
|   +-- GLEAM4.2a_monthly/        # Grid ET reference (NetCDF)
|   +-- GLEAM_hybrid_PLUMBER2/    # Station ET reference (per-station
    NetCDF)
|   +-- ILAMB/Latent_Heat/FLUXCOM, ILAMB/Sensible_Heat/FLUXCOM
|   +-- PLUMBER2/                 # Station energy-flux reference (per-
    station NetCDF)
|   +-- GLEAM_hybrid_PLUMBER2.csv, PLUMBER2.csv   # Station lists
`-- Simulation/Initial_test/
    +-- grid/                     # Grid simulation (grid_case_hist_*.nc)
    `-- stn/                      # Station simulation (per-station sim_
        *.nc) + station_case.csv
```

Note: Station references/simulations use a **station-list CSV** (with columns such as `ID`, longitude/latitude, `DIR`, etc.) to point each station to its corresponding NetCDF; `smoke-test` automatically rewrites the relative paths in the list to absolute paths on the local machine.

15.3 Configuration file

The `openbench-smoke.yaml` generated by `smoke-test` is a complete OpenBench configuration (the automatically filled-in absolute paths have been omitted):

```yaml
project:
  name: openbench_initial_test_smoke
  output_dir: <work_dir>/output
  years: [2004, 2005]
  tim_res: Month
  grid_res: 2.0
  weight: none
  num_cores: 1
  time_alignment: intersection
  unified_mask: true
  generate_report: false

evaluation:
  variables: [Evapotranspiration, Latent_Heat, Sensible_Heat]

reference:
  data_root: <reference_root>
```

```
  Evapotranspiration: [OpenBench_Smoke_GLEAM4_2a,
     OpenBench_Smoke_GLEAM_hybrid_PLUMBER2]
  Latent_Heat:        [OpenBench_Smoke_ILAMB_monthly,
     OpenBench_Smoke_PLUMBER2]
  Sensible_Heat:      [OpenBench_Smoke_ILAMB_monthly,
     OpenBench_Smoke_PLUMBER2]

simulation:
  grid_case:                        # Grid simulation
    model: CoLM
    data_type: grid
    root_dir: <sim>/grid
    prefix: grid_case_hist_
    variables:
      Evapotranspiration: {varname: f_fevpa, varunit: mm s-1}
      Latent_Heat:        {varname: f_lfevpa, varunit: W m-2}
      Sensible_Heat:      {varname: f_fsena, varunit: W m-2}
  station_case:                     # Station simulation
    model: CoLM
    data_type: stn
    root_dir: <sim>/stn
    fulllist: <work_dir>/lists/station_case.csv
    variables:
      Evapotranspiration: {varname: f_fevpa, varunit: mm s-1}
      Latent_Heat:        {varname: f_lfevpa, varunit: W m-2}
      Sensible_Heat:      {varname: f_fsena, varunit: W m-2}

metrics: [bias, RMSE, correlation]
scores:  [Overall_Score]
comparison: {enabled: true, items: [HeatMap]}
statistics: {enabled: false}
```

Key points:

- One variable can be bound to **multiple** references (the list syntax); OpenBench evaluates against each reference separately.
- The `varname`/`varunit` under `variables` map the variable names and units in the model files to the standard variables; units are converted automatically by OpenBench (e.g. `mm s-1` → `mm day-1`).
- This example uses `weight: none` (arithmetic mean) and `generate_report: false` (no PDF/HTML; only data and figures are produced).

15.4 Validation: openbench check

Before the actual run, validate the configuration and data availability:

```
openbench check openbench-smoke.yaml
```

Real output (excerpt):

```
Reference data (6 sources for 3 variables):
  [OK] Evapotranspiration → OpenBench_Smoke_GLEAM4_2a (grid, Month)
  [OK] Evapotranspiration → OpenBench_Smoke_GLEAM_hybrid_PLUMBER2 (stn,
     Day)
  [OK] Latent_Heat → OpenBench_Smoke_ILAMB_monthly (grid, Month)
  [OK] Latent_Heat → OpenBench_Smoke_PLUMBER2 (stn, Day)
  [OK] Sensible_Heat → OpenBench_Smoke_ILAMB_monthly (grid, Month)
  [OK] Sensible_Heat → OpenBench_Smoke_PLUMBER2 (stn, Day)
  Evaluation tasks: 12 (6 references × 2 simulations)

Simulation data (2 models):
  [OK] grid_case (model: CoLM, ...)
  [OK] station_case (model: CoLM, ...)

Metrics: bias, RMSE, correlation
Scores: Overall_Score
Options: Time alignment: intersection | Unified mask: True | Comparison:
   True

[OK] Config valid (3 variables, 6 references, 2 simulations). Ready to
   run.
```

OpenBench expands the 6 (variable, reference) combinations × 2 simulations into **12 evaluation tasks**.

15.5 Running: openbench run

```
openbench run openbench-smoke.yaml          # Incremental; only recompute
    parts affected by config changes
openbench run openbench-smoke.yaml --force  # Ignore the cache and
   recompute everything
```

The run completes preprocessing (regridding/clipping/unified mask), per-task evaluation, and comparison plotting in turn. The products land in `<work_dir>/output/openbench_initial_test_smoke/`:

```
output/openbench_initial_test_smoke/
+-- data/        # Preprocessed sim/ref NetCDF (grid + per-station)
+-- metrics/     # One NC (grid)/CSV (station) per (variable, reference,
   simulation, metric) + figures
```

```
+-- scores/      # NC/CSV for Overall_Score + figures
+-- comparisons/HeatMap/
|   +-- scenarios_Overall_Score_comparison.csv        # Summary score
    table
|   `-- scenarios_Overall_Score_comparison_heatmap.jpg
`-- .openbench_cache.json   # Incremental cache index
```

15.6 Interpreting the results

(1) Overall-score heat map. The comparison stage aggregates the `Overall_Score` of all (variable, reference) × simulation into a single heat map (`comparisons/HeatMap/scenarios_Overall_Scor`

Table 18: Initial_test overall scores (Overall_Score; closer to 1 is better)

Variable	Reference	grid_case	station_case
Evapotranspiration	GLEAM4_2a	0.391	0.462
Evapotranspiration	GLEAM_hybrid_PLUMBER2	0.748	0.741
Latent_Heat	ILAMB_monthly	0.658	0.586
Latent_Heat	PLUMBER2	0.748	0.772
Sensible_Heat	ILAMB_monthly	0.645	0.466
Sensible_Heat	PLUMBER2	0.704	0.718

(2) Per-grid / per-station products. A standard evaluation produces three kinds of visualization for each (variable, reference, simulation): a **grid spatial distribution map** (per-grid NetCDF + global/regional map, under `metrics/` and `scores/`), a **station spatial map** (station locations colored by the metric), and a **station time-series plot** (the obs vs sim curve over time for each station, under `data/stn_. . ./`). Station evaluation also produces a CSV with one row per station:

```
scenario,ID,sim_lon,sim_lat,...,use_syear,use_eyear,KGESS,RMSE,
    correlation,bias,Overall_Score
usgs-cb-1,AT-Neu,11.32,47.12,...,2004,2005,0.902,10.73,0.969,-2.764,0.893
usgs-cb-1,AU-How
    ,131.15,-12.50,...,2004,2005,0.700,34.82,0.882,-26.62,0.652
```

(The above is an excerpt from `scores/Latent_Heat_stn_OpenBench_Smoke_PLUMBER2_station_case_ev`

The **grid spatial distribution map** (Figure 4) shows the spatial pattern of a metric at a glance—where it is good, where the bias is large:

Station results have two views (Figure 5): the **station spatial map** shows "which stations are good", and the **station time-series plot** shows whether the single-station obs and sim agree

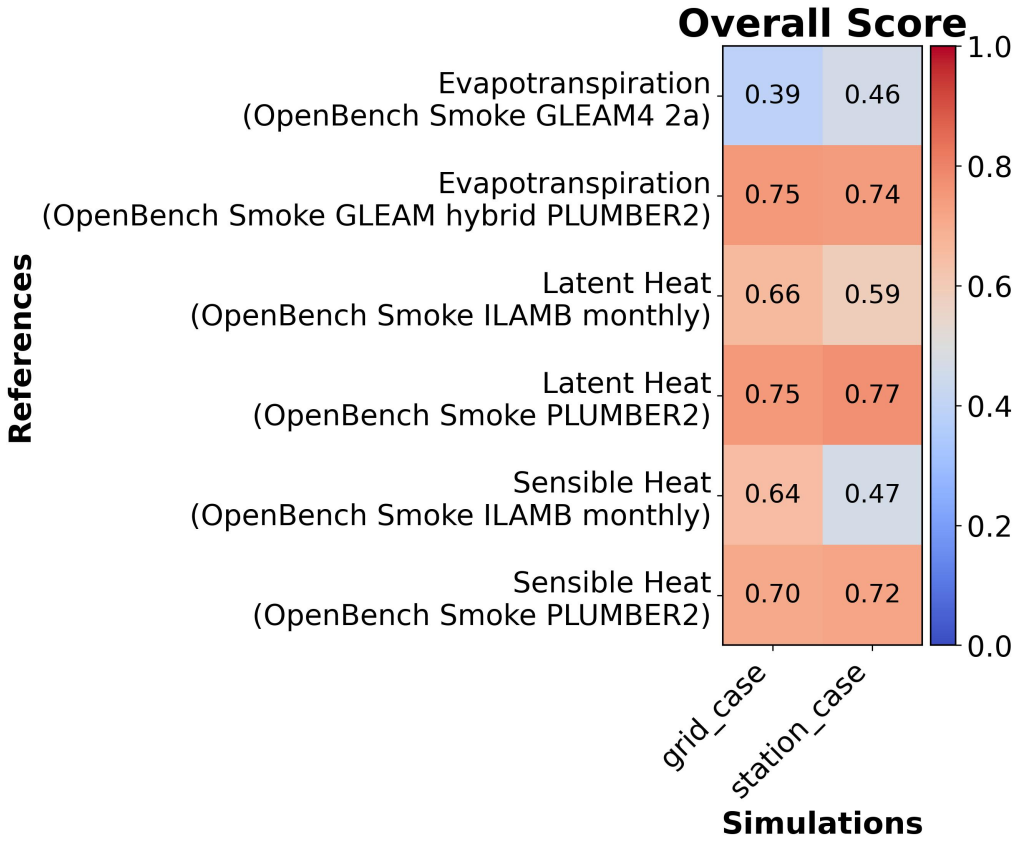


Figure 3: Overall-score heat map (`scenarios_Overall_Score_comparison_heatmap.jpg`).

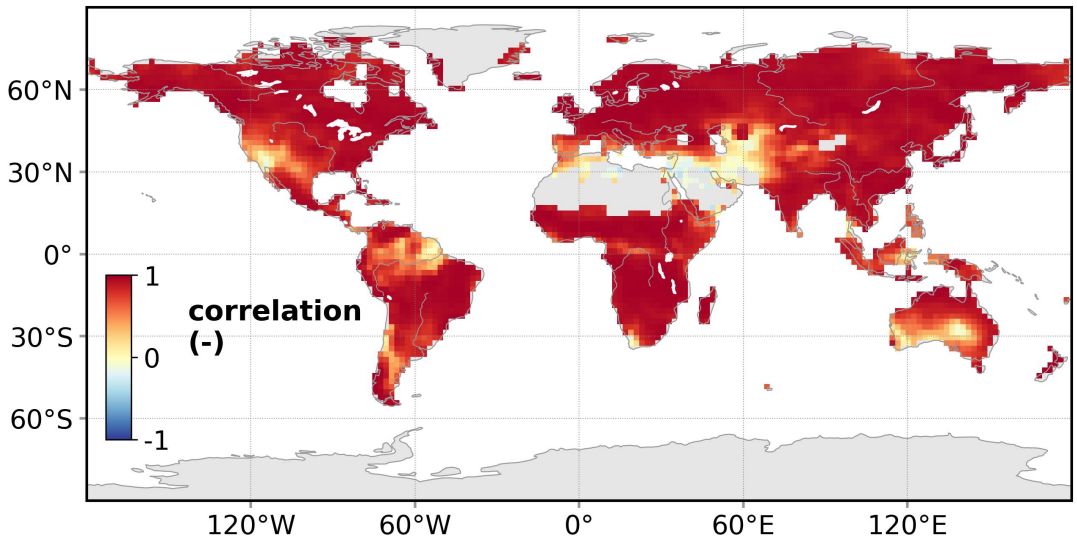
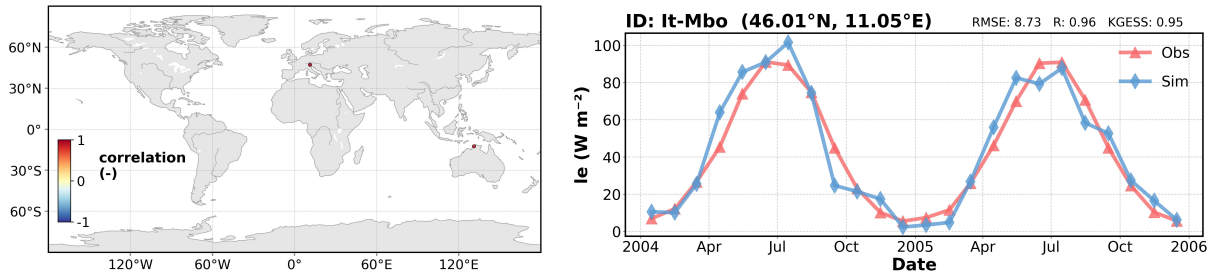


Figure 4: Example grid spatial distribution map: latent-heat correlation (CoLM `grid_case` vs ILAMB-FLUXCOM). Vegetated regions are generally highly correlated (red), while arid/transition zones are lower; gray indicates masked/no-data.

over time:



(a) Station spatial map: latent-heat correlation per station

(b) Station time series: obs vs sim at station IT-MBo (R=0.96)

Figure 5: Two views of station results: left is the station spatial distribution map, right is the single-station time-series plot (`data/stn_.../<var>__<ID>__timeseries.jpg`; red = observed, blue = simulated).

(3) Reading tips. Differences in score for the same variable against different references often reflect the uncertainty of the **reference data itself** (e.g. ET is low against GLEAM4_2a and high against GLEAM_hybrid_PLUMBER2); the difference between the grid and stn columns reflects the influence of the **evaluation mode** (regional grid vs station scale). To recompute or perform secondary analysis, simply read the NC/CSV files under `metrics/` and `scores/` with `xarray`/`pandas` (see Part V, Section 25).

> **Warning:** A full run on a clean machine may trigger Cartopy to download the Natural Earth basemap during plotting; in an offline environment, manually place the basemap beforehand as described in Section 5.2.2.

16 Example 2: Substitute Your Own Data

This example uses Example 1 as a skeleton and demonstrates replacing the reference and simulation with **your own data**. There are four steps: organize the data, register the reference, connect the model, and edit the configuration.

16.1 Step 1: Organize the data directory

We recommend keeping grid and station data separate, with one NetCDF per station for station data:

```
MyData/
+-- Reference/MyET/          # Reference: grid NetCDF or per-station
    NetCDF
`-- Simulation/MyModel/      # Simulation: model output NetCDF
```

16.2 Step 2: Register the reference dataset

Two approaches (written into the user overlay, leaving the package's built-in catalog untouched):

```
# Approach A: scan the directory for automatic detection
openbench ref scan /data/MyData/Reference

# Approach B: manual registration (specify root directory, variable names
    , units, etc.)
openbench ref register MyET --root-dir /data/MyData/Reference/MyET
openbench ref show MyET           # Confirm the variable mapping and
    paths
```

A **station reference** also needs a station-list CSV (with `ID`, longitude/latitude, and `DIR` pointing to each station's NetCDF):

```
openbench ref generate-station-list /data/MyData/Reference/MyET \
    -o /data/MyData/MyET_stations.csv
```

16.3 Step 3: Connect the model

If your model is not among the 22 built-in profiles, register one, or directly specify the variable names/units in the configuration via `variables`:

```
openbench model show CoLM2024     # Refer to the syntax of an existing
    profile
openbench model register MyModel  # Or register your own profile
```

16.4 Step 4: Edit the configuration and run

Replace `reference` and `simulation` in the Example 1 configuration with your own names and paths:

```
evaluation:
```

```
  variables: [Evapotranspiration]
reference:
  data_root: /data/MyData/Reference
  Evapotranspiration: MyET
simulation:
  my_run:
    model: MyModel            # Or a built-in profile name
    root_dir: /data/MyData/Simulation/MyModel
    data_type: grid
    variables:
      Evapotranspiration: {varname: ET, varunit: mm day-1}
```

```
openbench check my.yaml          # Always validate first: data reachable,
    variables resolvable
openbench run   my.yaml
```

Note: `check` before `run`: `check` reports, item by item, whether each (variable, reference) was located successfully and how many evaluation tasks were expanded in total, so it can catch path/variable-name/unit problems before the run.

17 Example 3: Extended Comparison Plots and Class Grouping

Building on Example 1, this example enables more comparison plots (Taylor, Portrait) and IGBP class grouping **by editing the configuration only**, demonstrating the rich products of the comparison stage. It can still be reproduced using the bundled sample.

17.1 Configuration changes

Make two changes to the Example 1 `openbench-smoke.yaml`:

```
project:
  IGBP_groupby: true                # Enable IGBP land-cover class
     grouping

comparison:
  enabled: true
  items: [HeatMap, Taylor_Diagram, Portrait_Plot_seasonal]   # Add two
     plot types
```

Then run as usual (`openbench run openbench-smoke.yaml --force`). The new products land in `comparisons/Taylor_Diagram/`, `comparisons/Portrait_Plot_seasonal/`, and `comparisons/IGBP_groupby/`.

17.2 Taylor diagram

The Taylor diagram plots the **correlation, normalized standard deviation, and CRMSD** of each (sim, ref) in the same polar coordinates, showing "who is closer to the reference" at a glance. The accompanying CSV gives the statistics, e.g. for Latent_Heat against PLUMBER2 in `station_case`: standard deviation 40.37, correlation 0.925, reference standard deviation 37.33; CRMSD has two columns—the **plotted** value 15.30 (back-derived from std/corr via the Taylor identity, to ensure the point falls on the contour) is used on the figure, while the CSV also records the **true mean** 16.41 (see the explanation of the two CRMSD columns in Example 5).

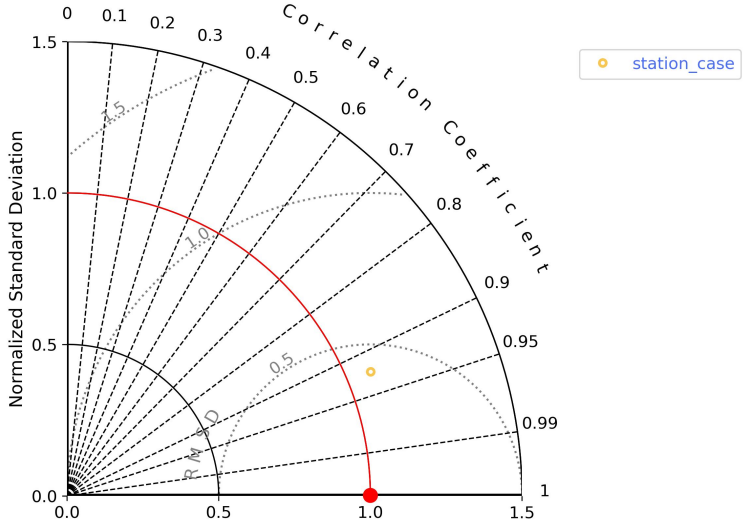


Figure 6: Taylor diagram for Latent_Heat against PLUMBER2 (`Taylor_Diagram__Latent_Heat__..._PLUMBER2.jpg`).

17.3 IGBP class grouping

With `IGBP_groupby` enabled, OpenBench uses the bundled IGBP mask to additionally aggregate the grid results **by the 17 land-cover classes**, producing per-class CSVs/heat maps (`comparisons/IGBP_groupby/<sim>__<ref>/`). The table below gives the per-class metrics for grid evapotranspiration against GLEAM4_2a (excerpt of real output; `N/A` means there are

no valid grid cells of that class within this sample domain):

Table 19: ET (grid_case vs GLEAM4_2a) metrics by IGBP class (excerpt)

IGBP class	bias	RMSE	correlation
evergreen_needleleaf_forest	−33.17	44.02	0.954
deciduous_broadleaf_forest	−47.25	57.34	0.932
mixed_forests	−36.10	48.43	0.956
grasslands	−29.94	36.13	0.888
croplands	−40.54	48.78	0.888
Overall	**−18.06**	**24.92**	**0.911**

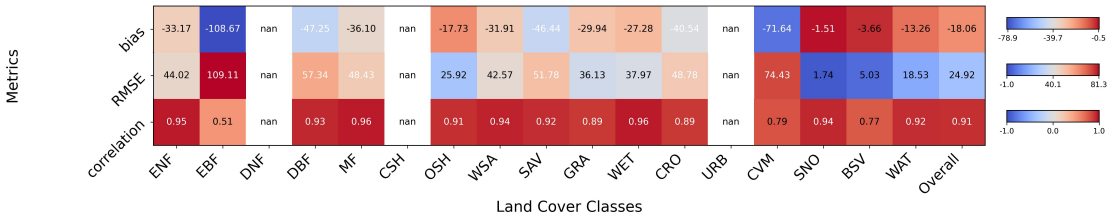


Figure 7: Heat map of ET metrics by IGBP class (`..._metrics_heatmap.jpg`).

Note: Per-class grouping helps locate "which land cover the model is most biased on". This sample is small demonstration data; the values only illustrate the structure and do not represent real model performance.

17.4 About Portrait and "completed with errors"

The Portrait (seasonal portrait) plot requires finite data for each (variable, reference, simulation) across all four seasons × each metric. This bundled sample has very little data, and `station_case` lacks finite values in some seasons, so the Portrait item raises an error and is skipped:

```
[X] Evaluation completed with errors
  - [comparison] Portrait_Plot_seasonal comparison failed:
    no finite data for Evapotranspiration/.../station_case/bias_DJF
```

This is caused by **insufficient data coverage**, not a defect: OpenBench reports the failed sub-item separately, while the other plot types (HeatMap, Taylor, IGBP grouping) are produced as usual; the command exits with a non-zero code to signal "some sub-items did not complete". On real data with complete coverage, Portrait will plot normally.

18 Example 4: Migrate from a Legacy Configuration and Run

If you already have a legacy v2.x configuration (JSON or Fortran NML), use `openbench migrate` to convert it into a new `openbench.yaml`:

```
# Legacy JSON configuration
openbench migrate old_config.json -o openbench.yaml

# Legacy Fortran NML configuration (requires the [migration] extra: pip
    install "colm-openbench[migration]")
openbench migrate main.nml -o openbench.yaml
```

Migration maps the legacy multi-file / NML fields into the new eight-block structure (project / evaluation / reference / simulation / ...). After conversion, **always validate before running**:

```
openbench check openbench.yaml     # Check the migration result:
    variables, reference bindings, path completeness
openbench run   openbench.yaml
```

> **Note:** After migration, we recommend reading through `openbench.yaml` manually: focus on the case of variable names, whether the references for each variable are all bound successfully, and whether the `root_dir` paths need adjustment for the new machine. The evaluation results of OpenBench 3.0 are numerically consistent with 2.x.

19 Example 5: Intercomparison of Several Real Land-Surface Models

This example is the most central use of OpenBench: comparing **several different land-surface models** under the same yardstick. We evaluate the latent heat (Latent_Heat) and sensible heat (Sensible_Heat) of four models—CoLM2024, CLM5, TE, GLDAS—all against the reference ILAMB-FLUXCOM, over the period 2004–2005, regridded to 2°. All values in this section are real run output.

19.1 Data and the challenge: the four models name things differently

The four models differ in native resolution, file naming, and variable names—which is exactly what must be connected model by model during configuration:

Table 20: Key differences among the four models (measured)

Model	Native resolution	File organization	LH variable name	SH variable name
CoLM2024	∼2°	one file per month	`f_lfevpa`	`f_fsena`
CLM5	∼2°	one file per month (h0 stream)	`EFLX_LH_TOT`	`FSH`
TE	0.5°	**one file per variable per year**	`LTNT`	`SENS`
GLDAS	0.25°	one file per month (NOAH)	`Qle_tavg`	`Qh_tavg`

19.2 Configuration: connect each model's file conventions

Each `simulation` entry uses `prefix`/`suffix`/`data_groupby` to describe that model's file naming, and uses `variables` to connect variable names and units. OpenBench searches for files by the pattern `{prefix}{year}*{suffix}.nc`:

```
simulation:
  CoLM2024:
    model: CoLM2024
    root_dir: /data/LSMs/CoLM2024
    data_type: grid
    data_groupby: Month
    prefix: Global_Grid_2x2_..._hist_     # File <prefix>YYYY-MM.nc
    variables:
      Latent_Heat:   {varname: f_lfevpa, varunit: W m-2}
      Sensible_Heat: {varname: f_fsena,  varunit: W m-2}
  CLM5:
    model: CLM5
    root_dir: /data/LSMs/CLM5
    data_groupby: Month
    prefix: IHistClm50BgcCropGSWP_CWD-1901-2014.clm2.h0.   # Take only
       the h0 stream
    variables:
      Latent_Heat:   {varname: EFLX_LH_TOT, varunit: W m-2}
      Sensible_Heat: {varname: FSH,         varunit: W m-2}
  TE:                                      # One file per variable ->
     put the token in the per-variable prefix
    model: TE
    root_dir: /data/LSMs/TE
    data_groupby: Year
```

```
    suffix: _GLB050
    variables:
      Latent_Heat:   {varname: LTNT, varunit: W m-2, prefix: "YEE2_JRA-55
          _LTNT_M"}
      Sensible_Heat: {varname: SENS, varunit: W m-2, prefix: "YEE2_JRA-55
          _SENS_M"}
  GLDAS:
    model: GLDAS
    root_dir: /data/LSMs/GLDAS
    data_groupby: Month
    prefix: GLDAS_NOAH025_M.A                # File ...A YYYYMM .021.nc
    suffix: ".021"                           # Note: a suffix starting
       with a dot must be quoted
    variables:
      Latent_Heat:   {varname: Qle_tavg, varunit: W m-2}
      Sensible_Heat: {varname: Qh_tavg,  varunit: W m-2}
```

Warning: Two pitfalls found in practice: (1) TE encodes the variable name in the *file name* → use a **per-variable prefix** (`variables.<var>.prefix`) to carry the token in; (2) the GLDAS suffix `.021` would be parsed by YAML as the number 0.021, so it must be written as `".021"`.

19.3 Validate and run

```
openbench check multi.yaml      # Expands to 8 tasks (2 variables × 4
   models)
openbench run   multi.yaml --force
```

OpenBench automatically regrids 0.25° (GLDAS), 0.5° (TE), and 2° (CoLM2024/CLM5) to the target 2° and then evaluates pair by pair, with no manual preprocessing required.

19.4 Result: overall-score heat map

The comparison stage aggregates the `Overall_Score` of the four models into a single table (real output):

Table 21: Overall scores of the four models (against ILAMB-FLUXCOM; closer to 1 is better)

Variable	CoLM2024	CLM5	TE	GLDAS
Latent_Heat	**0.724**	0.712	0.697	0.693
Sensible_Heat	0.687	**0.694**	0.610	0.566

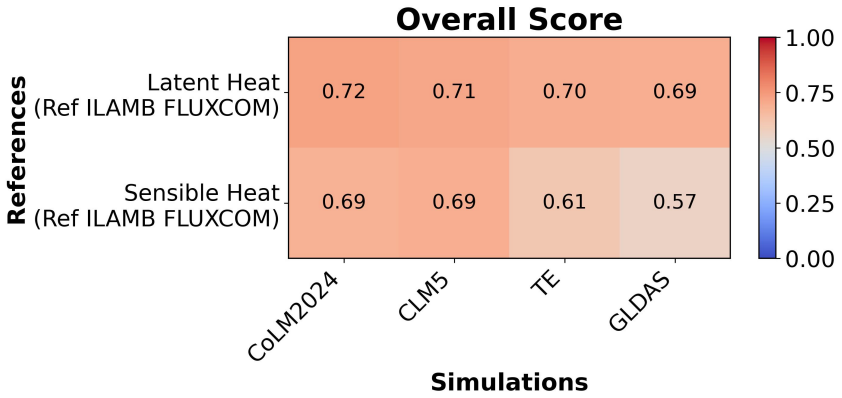


Figure 8: Overall-score heat map of the four models.

19.5 Result: Taylor diagram

The Taylor diagram plots the correlation, standard deviation, and CRMSD of the four models on a single figure for comparison (latent heat, real values). The table below lists the **raw** standard deviation; the figure uses the **normalized** standard deviation (= standard deviation ÷ reference standard deviation), i.e. the **radial distance** of the point from the origin (the reference point is at radial 1.0, correlation 1.0). For example, CoLM2024's radial value $= 20.13/24.89 = 0.81$.

Table 22: Latent-heat Taylor statistics (reference standard deviation = 24.89 W m^{-2})

Model	Std. dev.	Correlation	CRMSD (figure)	CRMSD (true mean)
CoLM2024	20.13	**0.898**	**11.17**	9.49
CLM5	14.91	0.855	14.41	11.01
TE	21.97	0.887	11.50	10.65
GLDAS	22.57	0.864	12.60	10.58

Note: The two CRMSD columns in the table mean different things: **CRMSD (figure)** is back-derived from the aggregated standard deviation and correlation via the Taylor identity CRMSD $= \sqrt{\sigma_s^2 + \sigma_r^2 - 2\sigma_s\sigma_r\, r}$; it is the *actual distance from the plotted point to the reference point*, ensuring the point falls exactly on the CRMSD contour. **CRMSD (true mean)** is the direct average of the per-grid/per-station CRMSD. The two differ because "averaging each per-point statistic separately" does not satisfy the Taylor identity—OpenBench therefore uses the former for plotting and the latter to record the true error magnitude (the columns `<sim>_RMS` and `<sim>_RMS_mean` in the CSV, respectively).

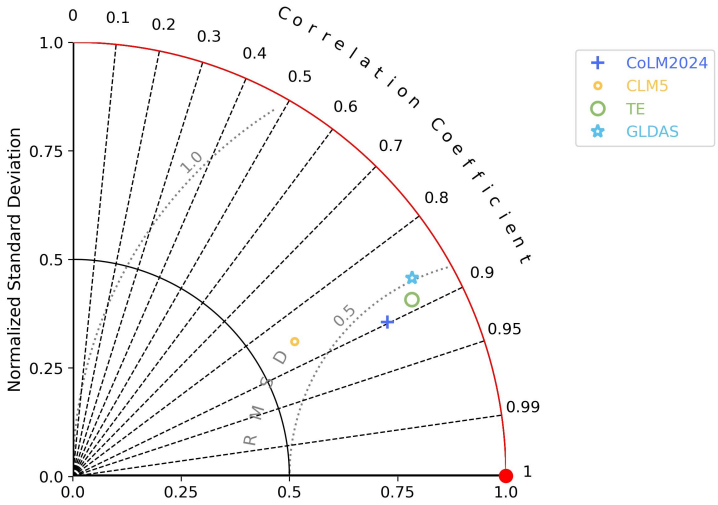


Figure 9: Latent-heat Taylor diagram (four models against ILAMB-FLUXCOM).

19.6 Reading tips

Combining the HeatMap and Taylor diagram: for latent heat CoLM2024 has the highest correlation (0.898) and the smallest plotted CRMSD (11.17), and leads on the overall score; CLM5 has a low standard deviation (variability too weak) but a small bias; sensible heat is overall harder to evaluate than latent heat (all models score lower, GLDAS lowest). This kind of **intercomparison** is exactly the value of OpenBench: one configuration, a unified standard, directly answering "which model is better on which flux".

> **Note:** To further pin down the source of the differences, you can add `IGBP_groupby` (view per-class performance by land cover, see Example 3) or more comparison plot types (Portrait, Parallel_Coordinates, etc., see Part III, Section 12.3).

20 Are the Results Reasonable: How to Read the Figures

20.1 Quickly locating results

First locate the file by "what I want to see" (paths relative to `output/<project_name>/`):

20.2 How to judge good vs bad

After producing results, here is how to **quickly judge good vs bad** and spot anomalies:

I want to see...	Where to go
Total score (overall score)	`comparisons/HeatMap/scenarios_Overall_Score_comparison_heatmap.jpg` (data `..._comparison.csv`)
Grid spatial distribution	`scores/<var>_..._Overall_Score.jpg`, `metrics/<var>_..._<metric>.jpg` (global/regional map)
Station spatial map	`metrics/<var>_stn_<R>_<S>_<metric>.jpg` (stations colored by metric)
Station time-series plot	`data/stn_<R>_<S>/<var>__<ID>__timeseries.jpg` (single-station obs vs sim curve)
Per-station metrics	`scores/<var>_stn_<R>_<S>_evaluations.csv` (also under `metrics/`; one row per station)
Per-grid metric data	`metrics/<var>_ref_<R>_sim_<S>_<metric>.nc`
Class grouping results	`comparisons/{IGBP,PFT,Climate_zone}_groupby/`
Comparison plots (Taylor, etc.)	`comparisons/<plot type>/`
Debug a failed run	`run.log` (records the success/failure and cause of each task by stage)

20.2.1 Scores

Normalized to $[0, 1]$, **closer to 1 is better**. `Overall_Score` is the composite score and is the core of multi-model ranking and the portrait; to look at one aspect alone, use `nBiasScore` (bias)/`nRMSEScore` (error), etc. Empirically, > 0.7 is good, 0.4–0.7 is fair, < 0.4 is fairly biased (but the absolute threshold varies by variable/reference; the emphasis is on *intercomparison*).

20.2.2 Metric direction

`bias`: look at sign and magnitude (0 is best; positive = overestimate); `RMSE`/`ubRMSE` smaller is better; `correlation` closer to 1 is better (temporal/spatial agreement); `KGE`/`NSE` ≤ 1, larger is better, < 0 means worse than just using the observed mean.

20.2.3 Taylor diagram

One point = one model. **Closer to the reference point (REF) is better**: the radial distance from the origin is the standard deviation (equal to the reference means the variability amplitude is right), the azimuth is the correlation (closer to the horizontal axis means higher correlation), and the distance to the reference point is the CRMSD (smaller is better). When comparing multiple models, the "point closest to REF" is the best.

20.2.4 HeatMap

Rows and columns are (variable/reference) × model, and the color shade is the score. A **blank/NaN** indicates that combination has *no result*—usually because the data does not overlap, the variable is missing, or that pair's evaluation failed (check `run.log` for the cause), not "a score of 0".

20.2.5 Grid spatial map

Look at the **spatial pattern** of the bias: whether some region is systematically high/low, whether land-sea boundaries/high latitudes are anomalous, whether there are large patches of NaN (mask or missing data). Combined with `IGBP_groupby`, this can locate "which land cover is worst".

20.2.6 Station scatter / CSV

Station evaluation has one row per station, with `correlation`/`KGESS`, etc. given per station; extreme values at individual stations are often a data-quality issue at that station, and can be removed using `min_year_threshold` or a custom filter.

> **Warning:** If something "looks very bad", rule out **convention issues** before concluding: whether the units were converted correctly (`varunit`), whether time/space is aligned, whether `weight` is as expected, whether a climatological mode is being used. These are more common than "the model is really bad".

Part 5 Developer Guide

This part is aimed at developers who need to **extend** OpenBench, **integrate** their own data/models, perform **downstream analysis**, or **contribute**. For deeper internal implementation details, consult the source code in `src/openbench/` directly.

21 Overview of Extension Points

The OpenBench source is organized into several subpackages. The extension points are concentrated in `data` (data/registry), `config` (configuration/migration), `core` (metrics/scores/statistics/visualization bridge), and `runner` (orchestration/caching).

Table 23: Subpackages of `src/openbench/` and Their Main Responsibilities

Subpackage	Responsibility / Common Extension Points
`cli`	Command-line entry points (run/check/ref/model/sim/cache/migrate/gui/smoke-test)
`config`	Configuration loading, schema, migration, legacy handling
`core`	Evaluation engine: metrics, scores, comparison, 17 statistics, visualization bridge
`data`	Data processing, registry (reference/model catalogs), caching, custom filters
`runner`	Task planning, parallel execution, cache fingerprinting, checkpoint/resume, post-processing
`visualization`	Plot types (`Fig_*`) and color tables
`remote`	SSH remote execution
`gui`	Optional graphical wizard

22 Registering Reference Data and Model Profiles

There are two ways to integrate your own reference data into the registry: **automatic registration via scanning** and **manual registration**.

```
# 1) Scan a directory; automatically detect and register available
   datasets
openbench ref scan /data/MyReference
```

Table 24: Common Tasks → Where to Make Changes

I want to…	Go to
Add **gridded** reference data	Section 22, "Gridded Reference Datasets (Entry Example)"
Add **station** reference data	Section 22, "Station Reference Datasets (Two Forms)"
Add a **new model** (profile)	Sections 22.8, 22.8.1 (structure example)
Walk through integration end-to-end	Section 22.6 (7-step verification) / Part IV, Case 2
Scan simulation output to generate config	`openbench sim scan` (Part III, Section 11.6)
Add a new metric/score	`core/metrics.py`, `core/scores.py` (Section 23)
Add new statistics/plots	`core/statistics/`, `visualization/` (Section 23.2)
Apply QC filtering to a reference	`data/custom/` filters (Section 23.3)
Downstream analysis of results	Read NC/CSV with `xarray`/`pandas` (Section 25)

```
# 2) Manually register/update a single dataset
openbench ref register MyET --root-dir /data/MyReference/MyET

# 3) Register a "reference profile" (variable-name/unit mapping template,
    reusable across datasets)
openbench ref register-profile ...

# Inspect and locate
openbench ref show MyET
openbench ref path MyET
```

Registration writes to a **user overlay** (under `.openbench/` in the user's home directory) and does not modify the package's built-in catalog, making it convenient for personal/team extensions.

22.1 Key Fields of a Reference Dataset

A reference entry must declare: `data_type` (grid/stn), `tim_res`, `grid_res`, `years`, `root_dir`, and for each variable the `varname` (variable name inside the NetCDF), `varunit` (used for unit conversion), and `prefix`/`suffix`/`sub_dir` (file location). A station dataset additionally requires a **station list CSV** (containing `ID`, longitude/latitude, and `DIR` pointing to each station's NetCDF), which can be generated automatically with `openbench ref generate-station-list`. For a complete reference overlay example, see Part IV, Case 5 (whose ILAMB/GLEAM references are integrated as overlays).

22.2 Gridded Reference Datasets (Entry Example)

Gridded references are declared with `data_type: grid`. The key elements are `grid_res` and the per-variable file location (`sub_dir`/`prefix`/`suffix`):

```
MyGridET:
  name: MyGridET
  category: Water
  data_type: grid
  tim_res: Month
  grid_res: 0.5
  years: [2001, 2010]
  root_dir: /data/MyReference          # can also use ${
     OPENBENCH_REF_ROOT}/...
  variables:
    Evapotranspiration:
      varname: et                       # variable name inside the NetCDF
      varunit: mm month-1               # original unit (auto-converted
         to the evaluation unit)
      sub_dir: ET/monthly               # subdirectory under root_dir
      prefix: ET_                        # file is <prefix><year><suffix
         >.nc
      suffix: _v1
```

22.3 Station Reference Datasets (Two Forms)

Station references can be organized in **two** ways; choose one according to the data form:

(a) One NetCDF per station + station list CSV (e.g. the PLUMBER2/FLUXNET style):

```
MyStnLH:
  name: MyStnLH
  category: Energy
  data_type: stn
  tim_res: Day
  data_groupby: single
  years: [2004, 2005]
  root_dir: /data/MyReference/MyStnLH
  fulllist: /data/MyReference/MyStnLH/stations.csv   # station list
  variables:
    Latent_Heat: {varname: Qle, varunit: W m-2}
```

The station list CSV has columns `ID,SYEAR,EYEAR,LON,LAT,Flag,DIR` (`DIR` points to each station's NetCDF), and can be generated automatically with `openbench ref generate-station-list <dir> -o stations.csv`.

(b) A single merged NetCDF (with a station dimension) + station_matching (e.g. a merged streamflow station file):

```
MyStreamflow:
  name: MyStreamflow
  category: Water
  data_type: stn
  tim_res: Month
  root_dir: /data/MyReference/Streamflow
  station_matching:
    dataset_file: streamflow_all.nc     # single merged file
    station_id_var: station             # station dimension/ID variable
    lat_var: Lat
    lon_var: Lon
    time_var: time
    discharge_var: Disch               # target variable
    area_var: upstream_area            # catchment area (used for
       streamflow matching)
    method: cama_allocation            # station-to-river-network
       matching method
```

22.4 Bulk Discovery and Registration with ref scan

When there are many datasets, registering them one by one with `ref register` is tedious. `openbench ref scan` can automatically discover and register a batch of reference datasets **based on a conventional directory structure**. The expected directory layout is:

```
# Grid: single-variable datasets (each dataset contains only one variable
    )
<REF_ROOT>/Grid/{LowRes,MidRes,HigRes}/<category>/<variable>/<dataset>/*.
    nc
# Grid: composite datasets (one NetCDF containing multiple variables)
<REF_ROOT>/Grid/{LowRes,MidRes,HigRes}/Composite/<dataset>/*.nc
# Station
<REF_ROOT>/Station/<category>/<variable>/<dataset>/*.nc
```

Every folder level has meaning, and the scanner infers metadata accordingly:

- `Grid` / `Station`: data type (`data_type: grid / stn`).
- `LowRes` / `MidRes` / `HigRes`: resolution tier (low/mid/high). A dataset spanning multiple tiers is grouped by base name and named `<dataset>_LowRes`, etc.; the GUI uses this to offer a resolution selection.

- `<category>`: category (e.g. `Water`, `Energy`, `Carbon`).
- `<variable>`: **standard variable name**—this folder name *is* the variable provided by the dataset, e.g. `Surface_Albedo`, `Latent_Heat`, `Evapotranspiration` (must match the names used in `evaluation.variables`).
- `Composite`: a special "category" placeholder—used when a single NetCDF **contains multiple variables** at once (in which case there are *no* per-variable folders); the scanner infers the mapping from the variables inside the file.
- `<dataset>`: dataset name (e.g. `GLEAM_v4.2a`).

The recommended approach is "preview first, register second":

```
openbench ref scan /data/Reference --dry-run     # only show what would
   be registered, write nothing
openbench ref scan /data/Reference --auto         # register in bulk once
    confirmed (-y)
openbench ref scan /data/Reference --rescan       # also refresh already-
   registered datasets
openbench ref scan /data/Reference --only "GLEAM*" # only process
   datasets matching the name
```

Scan results are written to the **user overlay** (the catalog under `.openbench/` in the home directory) and can then be viewed with `ref list` / `ref show`.

22.4.1 What to Do When the Scan Encounters Unrecognized Data

When a folder **does not conform to the conventional layout** (common causes: *variable ambiguity / multiple candidates* inside the NC, NC files *nested too deeply*, or an unrecognized directory structure), the scan lists it as `skipped` (printing the path and reason) and, during an interactive scan (including `openbench init`), offers you a **four-way choice**:

- `[s]` **Skip for now**: exclude these folders this time, register the rest as usual; the next scan will ask again.
- `[p]` **Create/update a profile and rescan**: interactively define a **reference profile** for this data (specifying layout and variable names/units) so OpenBench learns how to read it, then *automatically rescan* and register. Suitable for "the data is useful, the layout is just non-standard."

- `[i]` **Add an ignore profile and rescan**: **permanently mark this folder as ignored** (writing a `layout: ignore` profile), so future scans no longer prompt. Suitable for "irrelevant / non-data" folders.
- `[a]` **Abort**: cancel this scan.

A complete example. Suppose `/data/MyRef/Grid/MidRes/Water/Evapotranspiration/MyET/` contains *multiple* NetCDF subdirectories and the scanner cannot determine which one is the dataset (reason `ambiguous_nc_subdirectories`):

```
$ openbench ref scan /data/MyRef
Scanning /data/MyRef (dry run)...
[DRY RUN] Found 8 dataset(s) to register/update. No catalog changes made
    yet.
  - Grid/MidRes/Water/Evapotranspiration/MyET:
      ambiguous_nc_subdirectories
Skip 1 unsupported folder(s), add a profile, ignore them, or abort:
  [s] skip for now
  [p] create/update reference profile and rescan
  [i] add ignore profile and rescan
  [a] abort
Action [s]: p                          # choose p: build a reference
    profile for it
Create reference profile for Grid/MidRes/Water/Evapotranspiration/MyET
Profile name [MyET]: MyET              # press Enter to use the default
    name
  # then follow the prompts: layout, file glob, and varname/varunit for
      each variable
Updated reference profile: MyET
Rescanning...                          # automatic rescan
[DRY RUN] Found 9 dataset(s) to register/update.   # MyET is now
    recognized
```

Choosing `[p]` writes the new profile to the user overlay (`~/.openbench/.../reference_profiles.yaml`) and *automatically rescans*, after which MyET registers normally. If the folder is actually irrelevant (e.g. it holds documentation), choose `[i]` instead, which writes a `layout: ignore` entry so future scans no longer prompt.

If a folder **cannot even have a profile inferred**, OpenBench suggests using `[i]` to ignore, `[s]` to skip, or **registering manually**: `openbench ref register NAME --root-dir ...` (providing all fields directly), or first creating a template with `openbench ref register-profile` and then applying it.

Note: Non-interactive / batch (CI) scenarios: add `--allow-skip` to make the scan *automatically skip* unrecognized folders without blocking, while still registering the rest as usual; to first see which folders would be skipped without writing any changes, use `--dry-run`.

Note: The `[s]/[p]/[i]/[a]` options and `--allow-skip` above apply to "layout/variable not recognized" for the **reference** scan. "Not recognized" during a **model** scan (`openbench sim scan ... --model auto`) is a different matter—the *model profile cannot be inferred*. For how to handle that (`--model` to specify / `--register-model` to generate a draft / `--allow-unresolved` to let it pass for now), see Part III, Section 11.7.

22.5 Adding, Moving, or Deleting Reference Root Directories

Adding a new reference root (without affecting already-registered ones). The registry is **cumulatively merged**: scanning another new root, or registering a single dataset on its own, both *merge into* the existing user catalog while old entries are preserved as-is—OpenBench specifically prevents "scanning a new root from overwriting/deleting old entries."

```
openbench ref scan /new/ref/root              # merge the new root's
    datasets into the existing registry
openbench ref register MyDS --root-dir /elsewhere/MyDS   # a single
    dataset, any path
```

Each dataset records its own `root_dir` and may come from a different path; thus one configuration can use references under multiple roots simultaneously.

Note: Portability: datasets scanned under `OPENBENCH_REF_ROOT` are stored as `${OPENBENCH_REF_ROOT}/...` (changing machines only requires changing the environment variable); those outside it are stored with **absolute paths** (used as-is, but require editing when changing machines). It is recommended to set the **main** reference library as `OPENBENCH_REF_ROOT` and supplement scattered datasets with absolute paths.

Data has moved. Simply re-register the dataset to the new path to *overwrite* the old entry: `openbench ref register NAME --root-dir /new/path` (or `ref scan` the new root).

Original path deleted (entry invalid). If a reference's data directory has been deleted, its entry in the registry becomes a **dangling reference**: `openbench check` marks it as `[X]` (data

unavailable), and `openbench ref path NAME` shows the path it points to for verification. To clean up:

```
openbench ref path MyDS          # see where it points (confirm it is
    indeed gone)
openbench ref delete MyDS        # delete the entry from the user catalog
     (auto-backed up)
```

> **Warning:** `ref delete` only removes entries in the **user overlay**; it cannot remove **built-in** references. Built-in reference data comes from `OPENBENCH_REF_ROOT`—if that data is moved, **do not** delete the entry; instead point `OPENBENCH_REF_ROOT` to the data's new location. `ref scan`/`rescan` only incrementally *add/update*; they **do not** automatically delete entries that no longer exist. To clean up invalid entries in bulk, delete them one by one with `ref delete`, or edit `~/.openbench/references/reference_catalog.yaml` directly (a backup is made automatically before deletion).

22.6 End-to-End Integration and Verification Workflow

Follow the steps below to integrate your own reference data, **verifying step by step** to avoid "registered but check cannot find the data":

1. **Prepare the directory**: organize the data as `<root>/<sub_dir>/<prefix><year><suffix>.nc` (one NC per station for station data).
2. **Register**: `openbench ref scan /data/MyReference` (automatic) or `openbench ref register MyET --root-dir ...` (manual).
3. **Verify registration**: `openbench ref show MyET` to confirm the variable mapping/resolution/years are correct; `openbench ref path MyET` to confirm the resolved actual path exists. *What you can see here is what check/run can find.*
4. **(Station) Generate the list**: `openbench ref generate-station-list /data/MyReference/MyET -o stations.csv`.
5. **Write the YAML**: bind the variable to `MyET` under `reference` (with `data_root` pointing to its root).
6. **Validate**: `openbench check my.yaml`—check each (variable, reference) pair for `[OK]` one by one.

7. **Minimal run**: first run a single variable with `--variable` to verify the whole pipeline: `openbench run my.yaml --variable Evapotranspiration`.

   Warning: Common causes of "registered but check cannot find the data": (1) variable name case/spelling does not match `evaluation.variables`; (2) `prefix/suffix/sub_dir` does not match the actual file names (compare with `ref path`); (3) `years` does not overlap the file years; (4) `data_root` points to the wrong root directory; (5) the `DIR` in the station CSV is wrong/relative.

22.7 Other ref Management Commands

- `openbench ref register-profile NAME`: register a "reference profile" (variable-name/unit mapping template), reusable across multiple similar datasets.
- `openbench ref optimize NAME`: convert a dataset to **Zarr** to speed up reading (useful for repeated evaluation of large data).
- `openbench ref convert-old OLD.yaml OUT.yaml`: convert an old-format reference YAML to a v3 registry description.
- `openbench ref delete NAME`: delete a reference in the user overlay (does not affect the package's built-in catalog).

22.8 Registering and Customizing Model Profiles

A model profile describes a model's variable names, units, and file-naming conventions. Common commands:

```
openbench model list                  # list 22 built-in + user profiles
openbench model show CoLM2024         # view the variable mapping
openbench model register MyModel ... # register/update a user profile
openbench model export CoLM2024 -o colm.yaml   # export to a standalone
    YAML (as a template)
openbench model import my_model.yaml           # import into the user
    catalog
openbench model alias  ...            # create an alias
openbench model validate MyModel      # check completeness
openbench model status                # registry status (built-in/user,
    completeness)
openbench model path MyModel          # whether the profile is built-in or
     user-defined
```

```
openbench model rename OLD NEW       # rename a user profile
openbench model remove-var MyModel Latent_Heat   # remove one variable
   from the profile
openbench model delete MyModel       # delete a user profile
```

Once `simulation.<label>.model` in the configuration references a profile name, the variable file-name patterns and unit conversions are applied automatically; the `variables` field within the entry overrides them in place. For models that "split files by variable," use a per-variable `prefix` (see Part III, Section 11 and Part IV, Case 5).

22.8.1 Model Profile Structure and Example

A profile is simply a YAML describing "how this model names variables/files/units." Top-level fields: `name`, `description`, `data_type` (grid/stn), `tim_res`, `grid_res`, `data_groupby`, `time_offset` (optional), plus a `variables` mapping (each standard variable → `varname`/`varunit`):

```
MyModel:
  name: MyModel
  description: My land model historical run
  data_type: grid
  tim_res: Month
  grid_res: 0.5
  data_groupby: Month            # files split by month
  variables:
    Latent_Heat:        {varname: LH,  varunit: W m-2}
    Sensible_Heat:      {varname: SH,  varunit: W m-2}
    Evapotranspiration: {varname: ET,  varunit: mm s-1}
```

Register and verify:

```
openbench model import my_model.yaml     # or openbench model register
   MyModel ...
openbench model show MyModel             # confirm the variable mapping
openbench model validate MyModel         # check completeness
```

Then reference it in the configuration with `simulation.<label>.model: MyModel`; file-name prefixes/suffixes, etc., can be given in the profile or in the simulation entry (the entry overrides the profile). OpenBench performs unit conversion automatically (e.g. `mm s-1` → the evaluation unit).

Note: Quick start: `openbench model export CoLM2024 -o my_model.yaml` exports

a built-in profile as a template; rename it, change `varname`/`varunit`, then `import`. Any model not among the 22 built-in ones is integrated this way.

23 Adding Custom Metrics, Scores, Statistics, and Visualizations

Metrics and scores are defined in `core/metrics.py` and `core/scores.py`, respectively. The conventions for adding a metric:

- Add a method to the `metrics` class with a signature like `def my_metric(self, s, o)` (`s`=simulation, `o`=observation/reference, both `xarray.DataArray`).
- Compute along the `time` dimension with `xarray`, and **handle edge cases explicitly**: guard against division by zero, all-NaN, constant series, too few samples, etc. with `xr.where(...)`, returning NaN rather than crashing.
- Drop NaN pairwise to keep the valid masks of `s`/`o` aligned; keep the `ddof` of standard deviation/variance consistent across related metrics.
- Scores (`scores.py`) should be normalized to 0–1 (higher is better) so they can feed into `Overall_Score`.

```
# core/metrics.py (illustrative)
def my_bias_ratio(self, s, o):
    num = s.mean(dim="time") - o.mean(dim="time")
    den = o.mean(dim="time")
    return xr.where(den != 0, num / den, np.nan)
```

Once added, it can be referenced by method name in the configuration's `metrics`/`scores` lists. For consistency constraints (such as ddof and mask alignment), see the source in `core/metrics.py` and `core/scores.py`.

23.1 Testing and Validation After Adding

After adding a metric/score, do at least three things before committing:

1) Edge-case unit tests—write a **genuinely runnable** pytest covering normal/constant/all-NaN/insufficient-samples cases, with meaningful assertions (do not write tautological assertions like `assert ... or True`). The example below uses the built-in `correlation`/`bias` for demonstration; replace them with your new metric and adjust the assertions to its *expected* behavior:

```
# tests/test_my_metric.py
import numpy as np, xarray as xr
from openbench.core.metrics import metrics

def DA(values):
    return xr.DataArray(np.asarray(values, dtype=float), dims="time")

m = metrics()

def test_normal_inputs():
    s, o = DA([1, 2, 3, 4, 5]), DA([1.1, 1.9, 3.2, 3.8, 5.1])
    assert np.isfinite(m.correlation(s, o))      # normal input should
       give a finite value
    assert m.correlation(s, o) > 0.9             # highly correlated

def test_constant_series_returns_nan():
    const, o = DA([2, 2, 2, 2, 2]), DA([1, 2, 3, 4, 5])
    assert np.isnan(m.correlation(const, o))     # std=0 -> correlation
       undefined = NaN
    assert np.isfinite(m.bias(const, o))         # but bias is still
       computable

def test_all_nan_returns_nan():
    nan, o = DA([np.nan] * 5), DA([1, 2, 3, 4, 5])
    assert np.isnan(m.bias(nan, o))              # all NaN -> NaN, no
       exception

def test_insufficient_samples():
    s, o = DA([1, 2]), DA([1, 3])
    np.asarray(m.correlation(s, o))              # tiny n should not
       crash (a value or NaN is fine)
```

Run: `python -m pytest tests/test_my_metric.py -q`. Focus on covering: division by zero (std=0/zero denominator), all-NaN, constant series, too few samples (n<2), and empty arrays—all of these should **return NaN rather than raise an exception**.

2) Run tests and lint:

```
python -m pytest tests/test_core/ -q
ruff check src/ tests/
```

3) Confirm the cache fingerprint changes: after adding/changing metrics/scores source code, its content fingerprint should change so that old caches automatically become invalid. To verify: run `openbench run` on the same configuration twice; after changing the source code, the second run should **recompute** (rather than hit the old cache). If it does not recompute, check whether the change is within a module covered by the fingerprint; if necessary, use `--force` or delete `.openbench_cache.json`.

23.2 Adding Custom Statistics and Visualizations

23.2.1 Statistics Modules

Statistical methods live in `core/statistics/` (one method per `stat_*.py`, 17 in total). When adding one, follow the input/output conventions of the existing modules, then reference it by name in `statistics.items` (note that `statistics` requires `enabled: true` *and* explicitly listed items).

23.2.2 Plots

Plots live in `visualization/Fig_*.py`. When adding a new plot type, follow the existing implementations in the same directory and wire it into the comparison/report pipeline. Reusing the built-in base plotting (including the `cartopy` basemap wrappers) saves much more time than hand-coding map code; in offline environments, remember to prepare the Natural Earth basemaps first (see Part II, Section 5.2.2).

23.3 Custom Filters and Data Processing

OpenBench supports custom data filters (e.g. quality control/screening on a given reference), placed under `data/custom/` and referenced by name through a dynamic loading mechanism (`load_filter`) (a built-in example is `GEBA_filter.py`). Filters act on a dataset during the preprocessing stage and are well suited for logic such as "keep only samples/stations/time periods satisfying a condition," without modifying the core code.

24 Remote Execution, Caching, and Reproducibility

Warning: Remote execution is currently driven **only through the GUI**; the **command line is not yet implemented**: `openbench run --remote ...` will error out directly and suggest using `openbench gui` instead. This section describes the GUI remote workflow and the underlying mechanism.

24.1 Two Ways to Use HPC

1. **Run directly on the cluster (recommended, simplest)**: install OpenBench on the login/compute nodes and run `openbench run openbench.yaml` just like locally (or put it in a job script submitted via Slurm/PBS). This path **does not require** `[remote]`.
2. **Submit from local to remote over SSH (GUI)**: after installing `[remote]`, use `openbench gui` to configure the remote host, upload config/data, run remotely, and retrieve results.

24.2 Running in a Cluster Job Script (Minimal Slurm Workflow)

Step 1: Self-check on the login node first (confirm the environment is ready before submitting the job, so you do not discover a bad install after queuing):

```
source ~/openbench-env/bin/activate
openbench smoke-test --run        # built-in sample runs through =
   environment OK
```

Step 2: Prepare Cartopy basemaps in advance on offline machines. Compute nodes typically lack external network access, and plotting will break when fetching Natural Earth fails. On an *internet-connected* login node, place the basemaps in a **shared directory** for compute nodes to reuse (see Part II, Section 5.2.2):

```
export CARTOPY_DATA_DIR=/shared/cartopy
python -m cartopy.feature.download physical --output "$CARTOPY_DATA_DIR"
```

Step 3: Write the job script `run_openbench.slurm`:

```
#!/bin/bash
#SBATCH -J openbench
#SBATCH -N 1
```

```
#SBATCH -n 16
#SBATCH -t 04:00:00
#SBATCH -o openbench_%j.log

source ~/openbench-env/bin/activate
export HDF5_USE_FILE_LOCKING=FALSE          # required on NFS/shared
   disks to avoid file-lock errors
export CARTOPY_DATA_DIR=/shared/cartopy     # reuse shared basemaps;
   plots work even offline

openbench run openbench.yaml --cores 16
```

Step 4: Submit:

```
sbatch run_openbench.slurm
```

Note: PBS/LSF work the same way: replace `#SBATCH` with the corresponding directives; the three body lines (activate environment + two exports + `openbench run`) stay unchanged. When node memory is tight, reduce `--cores` or enable `dask` in the configuration and set `dask.memory_limit`.

24.3 GUI Remote (`[remote]`)

The `remote` subpackage provides SSH connections (`connections/ssh`), credentials (`credentials`), and remote storage and synchronization (`storage/sync`). In the GUI's remote page: configure hostname/port/user, choose key or password, set the remote working directory → upload config and (optionally) data → run remotely → retrieve the artifacts.

Note: Troubleshooting tips: (1) first confirm passwordless login with the system `ssh user@host` (the key has been added to `authorized_keys`); (2) the remote must already have the same version of OpenBench and dependencies installed; (3) set `HDF5_USE_FILE_LOCKING=FALSE` on shared disks; (4) when compute nodes lack external network access, pre-place the `CARTOPY_DATA_DIR` basemaps (see Part II, Section 5.2.2).

For large-volume evaluations, combine the `dask` configuration in Part III, Section 13 to enable distributed scheduling, and set `num_cores` and `dask.memory_limit` according to node memory.

24.4 Caching and Reproducibility Mechanisms

24.4.1 Two Layers of Cache

OpenBench has two layers of cache:

- **Incremental evaluation cache**: each evaluation task is cached with "configuration + algorithm version + input file signature" as the key (`runner/hashing.py`). A hit skips the task; a change in source code (the metrics/scores content fingerprint), configuration, or input files invalidates and recomputes it automatically. The index is stored in `.openbench_cache.json` *within each output directory*.
- **Regrid weight cache**: conservative-remapping weights are reused across runs and stored in a global cache directory (can be specified via `OPENBENCH_REGRID_WEIGHT_CACHE_DIR`).

In addition, output is written atomically + fsync'd, so a crash leaves no half-written valid file; post-processing has a gate, so if any task fails no partial summary is produced; and after an interruption, a rerun safely skips completed work and redoes unfinished work.

24.4.2 Managing the Cache with the CLI

```
openbench cache status --regrid          # view regrid weight cache
   status
openbench cache status --regrid --json   # machine-readable output
openbench cache clear  --regrid          # clear the regrid weight cache
```

The **incremental evaluation cache** is stored per run. To clear the cache for a particular run, delete the `.openbench_cache.json` in that output directory; or add `--force` at runtime to bypass the cache and recompute everything.

> **Note:** When you suspect "I changed it but it did not recompute": (1) confirm the change is to an input included in the fingerprint (config/input files/algorithm source); (2) in development mode, if you only changed source code but the package version number did not change, invalidation relies on the metrics/scores content fingerprint—if it still behaves oddly, just use `--force` or delete `.openbench_cache.json`.

25 Downstream Analysis: Reading Results Directly with Python

The evaluation artifacts are standard NetCDF / CSV and can be further processed directly with `xarray` / `pandas`:

```
python - <<'PY'
import xarray as xr, pandas as pd

# Grid: per-grid metric NetCDF (dimensions lat/lon or time/lat/lon)
ds = xr.open_dataset("output/<run>/metrics/<var>_ref_<R>_sim_<S>_KGE.nc")

# Station: evaluation CSV with one row per station
df = pd.read_csv("output/<run>/metrics/<var>_stn_<R>_<S>_evaluations.csv"
    )

# Overall score summary (comparison stage, tab-separated)
heat = pd.read_csv(
    "output/<run>/comparisons/HeatMap/scenarios_Overall_Score_comparison.
       csv",
    sep="\t")
print(ds, df.head(), heat)
PY
```

This lets you feed OpenBench's standardized artifacts into your own downstream analysis, plotting, or paper workflow.

26 Contributing

After installing a development environment from source (see Part II):

```
python -m pytest -q              # run tests
ruff check src/ tests/          # lint
python -m build                 # build wheel + sdist
```

Contribution workflow: develop on a feature branch → pass tests and lint → open a Pull Request on GitHub. New features should come with tests and documentation; see `CONTRIBUTING` in the repository for details.